

Ex. No.: 01

DDL and DML commands

Date:

AIM:

To create a database table, add constraints (primary key, unique, check, Not null), insert rows, update and delete rows using SQL DDL and DML commands.

DESCRIPTION:

Data Definition Language:

- DDL statements create, change, and remove data structures from the database.
- The keywords CREATE, ALTER, DROP, RENAME, and TRUNCATE begin DDL statements.

Data Manipulation Language:

- DML statements begin with INSERT, SELECT, UPDATE, DELETE, or MERGE and are used to modify the table data by entering new rows, changing existing rows, removing existing rows as well as displaying columns and their data.

SYNTAX:

Data Definition Language (DDL):

Create Command:

```
create table <table_name>(
Column_name1 datatype [default expression or constraint],
Column_name2 datatype [default expression],
...
);
```

Alter Command:

```
alter table table_name add (column_name definition);
```

Rename Command:

To rename a table:

```
RENAME old_table_name TO new_table_name;
```

To rename a column:

```
ALTER TABLE table_name RENAME COLUMN old_column_name TO
new_column_name;
```

Drop Command:

To drop a table:

```
DROP TABLE table_name;
```

To drop a column:

```
ALTER TABLE table_name DROP COLUMN column_name;
```

Truncate Command:

```
TRUNCATE TABLE table_name;
```

Data Manipulation Language (DML):**Insert Command:****INSERT with column list:**

```
INSERT INTO table_name (column1, column2, ..., columnN)  
VALUES (value1, value2, ..., valueN);
```

INSERT without column list :

```
INSERT INTO table_name  
VALUES (value1, value2, ..., valueN);
```

INSERT multiple rows:

```
INSERT INTO table_name (column1, column2, ..., columnN)  
VALUES  
    (value1_1, value1_2, ..., value1_N),  
    (value2_1, value2_2, ..., value2_N),  
    ...  
    (valueM_1, valueM_2, ..., valueM_N);
```

Select Command:

```
SELECT <column1>, <column2> (or) * FROM table_name;
```

Update Command:

```
UPDATE table_name SET column_name = value WHERE condition;
```

SQL QUERY AND ITS OUTPUT :

```
SQL> CREATE TABLE D_EMPLOYEES (  
    EMP_ID NUMBER(5,0) PRIMARY KEY,  
    FIRST_NAME VARCHAR2(25) NOT NULL,  
    LAST_NAME VARCHAR2(30) NOT NULL,  
    EMAIL VARCHAR2(50) UNIQUE,  
    HIRE_DATE DATE DEFAULT SYSDATE,  
    SALARY NUMBER(8,2) CHECK (SALARY > 0),  
    DEPARTMENT VARCHAR2(30)  
);
```

Table Created

```
SQL> DESC D_EMPLOYEES
```

Name	Null?	Type
EMP_ID	NOT NULL	NUMBER(5)
FIRST_NAME	NOT NULL	VARCHAR2(25)
LAST_NAME	NOT NULL	VARCHAR2(30)
EMAIL	UNIQUE	VARCHAR2(50)
HIRE_DATE		DATE
SALARY		NUMBER(8,2)
DEPARTMENT		VARCHAR2(30)

```
SQL> INSERT INTO D_EMPLOYEES (EMP_ID, FIRST_NAME, LAST_NAME, EMAIL,  
SALARY, DEPARTMENT)  
VALUES (2, 'Jane', 'Smith', 'jane.smith@example.com', 55000, 'HR');
```

1 row Inserted

```
SQL> INSERT INTO D_EMPLOYEES (EMP_ID, FIRST_NAME, LAST_NAME, EMAIL,
SALARY, DEPARTMENT)
VALUES (3, 'Mike', 'Johnson', 'mike.johnson@example.com', 52000, 'Finance');
```

1 row Inserted

```
SQL> SELECT * FROM D_EMPLOYEES;
```

EMP_ID	FIRST_NAME	LAST_NAME	EMAIL	HIRE_DATE	SALARY	DEPARTMENT
--------	------------	-----------	-------	-----------	--------	------------

1	John	Doe	john.doe@example.com	2024-06-25	52000	IT
2	Jane	Smith	jane.smith@example.com	2024-06-25	55000	HR

```
SQL> UPDATE D_EMPLOYEES
SET SALARY = 52000
WHERE DEPARTMENT = 'IT';
```

1 row Updated

```
SQL> DELETE FROM D_EMPLOYEES
WHERE DEPARTMENT = 'Finance';
```

0 row Deleted

```
SQL> SELECT * FROM D_EMPLOYEES;
```

```
SQL> ALTER TABLE D_EMPLOYEES ADD (MANAGER_ID NUMBER(5,0));
```

```
SQL> DESC D_EMPLOYEES
```

Name	Null?	Type
EMP_ID	NOT NULL	NUMBER(5)
FIRST_NAME	NOT NULL	VARCHAR2(25)
LAST_NAME	NOT NULL	VARCHAR2(30)
EMAIL	UNIQUE	VARCHAR2(50)
HIRE_DATE		DATE
SALARY		NUMBER(8,2)
DEPARTMENT		VARCHAR2(30)

```
SQL> UPDATE D_EMPLOYEES SET MANAGER_ID = 1 WHERE EMP_ID = 2;
1 row updated
```

```
SQL> SELECT * FROM D_EMPLOYEES;
```

```
EMP_ID FIRST_NAME LAST_NAME EMAIL          HIRE_DATE  SALARY
DEPARTMENT MANAGER_ID
```

```
-----
1   John    Doe    john.doe@example.com  2024-06-25  52000 IT
2   Jane    Smith  jane.smith@example.com 2024-06-25  55000 HR      1
```

Result:

Thus the creation of a database table, adding constraints (primary key, unique, check, Not null), inserting rows, update and delete rows using SQL DDL and DML commands were executed successfully and its outputs verified.

Ex. No.: 2

Referential Integrity Constraints

Date:

AIM:

To create a set of tables, add foreign key constraints and incorporate referential integrity.

DESCRIPTION:

Referential Integrity Constraints:

- Referential integrity constraints can specify particular actions to be performed on the dependent rows in a child table if a referenced parent key value is modified.

SYNTAX:

PRIMARY KEY:

```
CREATE TABLE parent_table (  
    parent_id INT PRIMARY KEY,  
    column2 datatype NOT NULL,  
    column3 datatype DEFAULT value,  
    ...  
);
```

FOREIGN KEY:

```
CREATE TABLE child_table (  
    child_id INT PRIMARY KEY,  
    parent_id INT,  
    column2 datatype CHECK (condition),  
    column3 datatype UNIQUE,  
    ...  
    CONSTRAINT fk_parent FOREIGN KEY (parent_id) REFERENCES  
parent_table(parent_id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

SQL QUERY AND ITS OUTPUT :

```
SQL> CREATE TABLE D_DEPARTMENTS (  
    DEPT_ID NUMBER(5,0) PRIMARY KEY,  
    DEPT_NAME VARCHAR2(30) NOT NULL,  
    MANAGER_ID NUMBER(5,0)  
);
```

Table created.

```
SQL> DESC D_DEPARTMENTS
```

Name	Null?	Type

DEPT_ID	NOT NULL	NUMBER(5)
DEPT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(5)

```
SQL> ALTER TABLE D_EMPLOYEES  
ADD CONSTRAINT FK_EMP_DEPT  
FOREIGN KEY (DEPARTMENT) REFERENCES  
D_DEPARTMENTS(DEPT_NAME);
```

Table altered.

```
SQL> ALTER TABLE D_EMPLOYEES  
ADD CONSTRAINT FK_EMP_MANAGER  
FOREIGN KEY (MANAGER_ID) REFERENCES  
D_EMPLOYEES(EMP_ID);
```

Table altered.

```
SQL> INSERT INTO D_DEPARTMENTS (DEPT_ID, DEPT_NAME,  
MANAGER_ID)  
VALUES (1, 'IT', 1);
```

1 row inserted

```
SQL> INSERT INTO D_DEPARTMENTS (DEPT_ID, DEPT_NAME,  
MANAGER_ID)  
VALUES (2, 'HR', 2);
```

1 row inserted

```
SQL> SELECT constraint_name, table_name, r_constraint_name  
FROM user_constraints  
WHERE constraint_type = 'R' AND table_name IN ('D_EMPLOYEES',  
'D_DEPARTMENTS');
```

```
CONSTRAINT_NAME TABLE_NAME R_CONSTRAINT_NAME
```

```
-----  
FK_EMP_DEPT    D_EMPLOYEES SYS_C0011052  
FK_EMP_MANAGER D_EMPLOYEES SYS_C0011051
```

```
SQL> SELECT E.EMP_ID, E.FIRST_NAME, E.LAST_NAME,  
D.DEPT_NAME, M.FIRST_NAME AS MANAGER_NAME  
FROM D_EMPLOYEES E  
JOIN D_DEPARTMENTS D ON E.DEPARTMENT = D.DEPT_NAME  
LEFT JOIN D_EMPLOYEES M ON E.MANAGER_ID = M.EMP_ID;
```

```
EMP_ID FIRST_NAME LAST_NAME DEPT_NAME MANAGER_NAME
```

```
-----  
1   John    Doe    IT  
2   Jane    Smith  HR      John
```

Result:

Thus the creation of a set of tables, adding foreign key constraints and incorporating referential integrity were successfully executed and its outputs verified.

Ex. No.: 3

Where Clause and Aggregate Functions

Date:

AIM:

To query the database tables using different 'where' clause conditions and also implement aggregate functions.

DESCRIPTION:

WHERE clause:

- Retrieves specific columns from a table.
- The WHERE clause filters rows based on specified conditions.
- Can use comparison operators (=, <, >, <=, >=, !=) and logical operators (AND, OR, NOT).

Aggregate Functions:

a) AVG():

- Calculates the average value of a numeric column.
- Ignores NULL values in its calculation.

b) SUM() with GROUP BY:

- Calculates the sum of values in a column.
- GROUP BY clause groups the result by another column.
- Useful for summarizing data across categories.

c) MIN():

- Finds the minimum value in a column.
- Works with numeric, string, and date data types.

SYNTAX:

Where Statements:

SELECT column1, column2

FROM table_name

WHERE condition;

Aggregate Functions:

Average:

```
SELECT AVG(column_name) FROM table_name;
```

Sum of all values:

```
SELECT column1, SUM(column_name)
FROM table_name
GROUP BY column1;
```

Minimum of selected values:

```
SELECT MIN(column_name) FROM table_name;
```

Maximum of selected values:

```
SELECT MAX(column_name) FROM table_name;
```

Number of Entries in Column:

```
SELECT COUNT(column_name) FROM table_name;
```

SQL QUERY AND ITS OUTPUT :

```
SQL> SELECT FIRST_NAME, LAST_NAME
FROM D_CLIENTS
WHERE CLIENT_NUMBER = 5922;
```

```
FIRST_NAME LAST_NAME
```

```
-----
```

```
Hiram      Peters
```

```
SQL> SELECT TITLE, PRODUCER
FROM D_CDS
WHERE YEAR = '2000';
```

```
TITLE          PRODUCER
```

Party Music for All Occasions The Music Man
Carpe Diem R & B Inc.

```
SQL> SELECT EVENT_ID, SONG_ID, COMMENTS
FROM D_PLAY_LIST_ITEMS
WHERE COMMENTS IS NOT NULL;
```

EVENT_ID	SONG_ID	COMMENTS
100	45	Play late
100	47	Play early
105	48	Play after cake cutting
105	49	Play first
105	47	Play for the father

```
SQL> SELECT AVG(COST)
FROM D_EVENTS;
```

AVG(COST)
9000

```
SQL> SELECT VENUE_ID, SUM(COST)
FROM D_EVENTS
GROUP BY VENUE_ID;
```

VENUE_ID	SUM(COST)
100	8000
220	10000

```
SQL> SELECT MIN(COST)
FROM D_EVENTS;
```

MIN(COST)

8000

```
SQL> SELECT MAX(COST)
FROM D_EVENTS;
```

MAX(COST)

10000

```
SQL> SELECT COUNT(CLIENT_NUMBER)
FROM D_CLIENTS;
```

COUNT(CLIENT_NUMBER)

2

Result:

Thus the SQL commands to query the database tables using different 'where' clause conditions and to perform the aggregate operations were executed successfully and their outputs were verified.

Ex. No.: 4

Subqueries and Simple Join Operations

Date:

AIM:

To query the database tables and explore sub queries and simple join operations.

DESCRIPTION:

Subqueries:

- A query nested inside another query.
- Must be enclosed in parentheses.
- Can return a single value, a single row, a single column, or a table.
- Used in various parts of SQL statements (SELECT, FROM, WHERE, HAVING).
- Rules:
 - a) Must be enclosed within parentheses.
 - b) In comparison operations, must return a single column unless the main query is set up for multi-column comparison.

JOIN Operations:

- Used to combine rows from two or more tables based on a related column between them.
- The ON keyword specifies the join condition.

Types of JOINS:

INNER JOIN:

- Returns only the matching rows from both tables.

sql

```
SELECT * FROM table1
```

```
INNER JOIN table2 ON table1.column = table2.column;
```

LEFT (OUTER) JOIN:

- Returns all rows from the left table and matching rows from the right table.

- If no match, NULL values are returned for right table columns.

sql

```
SELECT * FROM table1
```

```
LEFT JOIN table2 ON table1.column = table2.column;
```

RIGHT (OUTER) JOIN:

- Returns all rows from the right table and matching rows from the left table.

- If no match, NULL values are returned for left table columns.

sql

```
SELECT * FROM table1
```

```
RIGHT JOIN table2 ON table1.column = table2.column;
```

FULL (OUTER) JOIN:

- Returns all rows when there's a match in either left or right table.

- If no match, NULL values are returned for columns from the table without a match.

sql

```
SELECT * FROM table1
```

```
FULL OUTER JOIN table2 ON table1.column = table2.column;
```

SYNTAX:

Subqueries:

```
SELECT column_name2, column_name3
```

```
FROM table1
```

```
WHERE column_name1 OPERATOR
```

```
(SELECT column_name1
```

```
FROM table2
```

```
WHERE condition);
```

Inner Join:

```
SELECT columns
FROM table1
JOIN table2
ON table1.column = table2.column;
```

Left Outer Join:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

Right Outer Join:

```
SELECT columns
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

Full Outer Join:

```
SELECT columns
FROM table1
FULL OUTER JOIN table2
ON table1.column = table2.column;
```

SQL QUERY AND ITS OUTPUT :

```
SQL> SELECT *
FROM D_EVENTS
WHERE VENUE_ID IN (SELECT ID FROM D_VENUES WHERE
LOC_TYPE = 'Private Home');
```

ID	NAME	EVENT_DATE	DESCRIPTION	COST
VENUE_ID	PACKAGE_CODE	THEME_CODE	CLIENT_NUMBER	

100 Peters Graduation 2004-05-14 Party for 200, red, white, blue... 8000
100 112 200 5922

```
SQL> SELECT C.CLIENT_NUMBER, C.FIRST_NAME, C.LAST_NAME
FROM D_CLIENTS C
WHERE EXISTS (
  SELECT 1
  FROM D_EVENTS E
  WHERE E.CLIENT_NUMBER = C.CLIENT_NUMBER AND E.COST >
9000
);
```

CLIENT_NUMBER	FIRST_NAME	LAST_NAME
---------------	------------	-----------

6133	Lauren	Vigil
------	--------	-------

```
SQL> SELECT E.NAME, V.ADDRESS
FROM D_EVENTS E
JOIN D_VENUES V ON E.VENUE_ID = V.ID;
```

NAME	ADDRESS
------	---------

Peters Graduation	52 West End Drive, Los Angeles, CA 90210
Vigil wedding	200 Pennsylvania Ave, Washington D.C. 09002

```
SQL> SELECT E.NAME, C.FIRST_NAME, C.LAST_NAME, V.ADDRESS
FROM D_EVENTS E
JOIN D_CLIENTS C ON E.CLIENT_NUMBER = C.CLIENT_NUMBER
JOIN D_VENUES V ON E.VENUE_ID = V.ID;
```

NAME	FIRST_NAME	LAST_NAME	ADDRESS
------	------------	-----------	---------

Peters Graduation Hiram Peters 52 West End Drive, Los Angeles, CA
90210

Vigil wedding Lauren Vigil 200 Pennsylvania Ave, Washington D.C.
09002

Result:

Thus the SQL commands to query the database tables, exploring the sub queries and simple join operations were successfully executed and its outputs verified.

Ex. No.: 5

Natural, Equi and Outer joins

Date:

AIM:

To query the database tables and explore natural, equi and outer joins.

DESCRIPTION:

Natural Join:

- SELECT column_list: Specifies the columns to retrieve from the joined result.
- table1 and table2: The tables to be joined.
- NATURAL JOIN: Automatically joins tables based on all columns with matching names.
- No explicit join condition is needed as it uses all common columns.

Equi Join:

- SELECT column_list: Specifies columns to retrieve from both tables.
- table1 and table2: The tables to be joined.
- JOIN ... ON: Specifies an explicit equality condition for joining.
- table1.column_name = table2.column_name: The join condition, matching values in specified columns.

Left Outer Join:

- SELECT column_list: Specifies columns to retrieve from both tables.
- table1 and table2: The tables to be joined, where table1 is the "left" table.
- LEFT JOIN: Returns all rows from the left table and matching rows from the right table.
- ON: Specifies the join condition.
- If no match in table2, NULL values are returned for its columns.

Right Outer Join:

- SELECT column_list: Specifies columns to retrieve from both tables.

- table1 and table2: The tables to be joined, where table2 is the "right" table.
- RIGHT JOIN: Returns all rows from the right table and matching rows from the left table.
- ON: Specifies the join condition.
- If no match in table1, NULL values are returned for its columns.

Syntax:

Natural Join

```
SELECT column_list  
FROM table1  
NATURAL JOIN table2;
```

Equi Join

```
SELECT column_list  
FROM table1  
JOIN table2 ON table1.column_name = table2.column_name;
```

Left Outer Join

```
SELECT column_list  
FROM table1  
LEFT JOIN table2 ON table1.column_name = table2.column_name;
```

Right Outer Join

```
SELECT column_list  
FROM table1  
RIGHT JOIN table2 ON table1.column_name = table2.column_name;
```

SQL QUERY AND ITS OUTPUT :

```
SQL> SELECT E.ID, E.NAME AS EVENT_NAME, C.FIRST_NAME,
C.LAST_NAME, V.ADDRESS
FROM D_EVENTS E
INNER JOIN D_CLIENTS C ON E.CLIENT_NUMBER =
C.CLIENT_NUMBER
INNER JOIN D_VENUES V ON E.VENUE_ID = V.ID;
```

ID	EVENT_NAME	FIRST_NAME	LAST_NAME	ADDRESS
100	Peters Graduation	Hiram	Peters	52 West End Drive, Los Angeles, CA 90210
105	Vigil wedding	Lauren	Vigil	200 Pennsylvania Ave, Washington D.C. 09002

```
SQL> SELECT E.ID, E.NAME AS EVENT_NAME, C.FIRST_NAME,
C.LAST_NAME, V.ADDRESS
FROM D_EVENTS E
LEFT OUTER JOIN D_CLIENTS C ON E.CLIENT_NUMBER =
C.CLIENT_NUMBER
LEFT OUTER JOIN D_VENUES V ON E.VENUE_ID = V.ID;
```

ID	EVENT_NAME	FIRST_NAME	LAST_NAME	ADDRESS
100	Peters Graduation	Hiram	Peters	52 West End Drive, Los Angeles, CA 90210
105	Vigil wedding	Lauren	Vigil	200 Pennsylvania Ave, Washington D.C. 09002

```
SQL> SELECT E.ID, E.NAME AS EVENT_NAME, C.FIRST_NAME,
C.LAST_NAME, V.ADDRESS
FROM D_EVENTS E
RIGHT OUTER JOIN D_CLIENTS C ON E.CLIENT_NUMBER =
C.CLIENT_NUMBER
```

RIGHT OUTER JOIN D_VENUES V ON E.VENUE_ID = V.ID;

ID	EVENT_NAME	FIRST_NAME	LAST_NAME	ADDRESS
100	Peters Graduation	Hiram	Peters	52 West End Drive, Los Angeles, CA 90210
105	Vigil wedding	Lauren	Vigil	200 Pennsylvania Ave, Washington D.C. 09002
				4 Mahogany Drive, Boston, MA 10010
				87 Park Avenue, San Diego, CA 28978
				4 Primrose Lane, Chevy Chase, MD 22127
				123 Magnolia Road, Hudson, N.Y. 11220

```
SQL> SELECT E.ID, E.NAME AS EVENT_NAME, C.FIRST_NAME,
C.LAST_NAME, V.ADDRESS
FROM D_EVENTS E
FULL OUTER JOIN D_CLIENTS C ON E.CLIENT_NUMBER =
C.CLIENT_NUMBER
FULL OUTER JOIN D_VENUES V ON E.VENUE_ID = V.ID;
```

ID	EVENT_NAME	FIRST_NAME	LAST_NAME	ADDRESS
100	Peters Graduation	Hiram	Peters	52 West End Drive, Los Angeles, CA 90210
105	Vigil wedding	Lauren	Vigil	200 Pennsylvania Ave, Washington D.C. 09002
		Serena	Jones	4 Mahogany Drive, Boston, MA 10010
				87 Park Avenue, San Diego, CA 28978
				4 Primrose Lane, Chevy Chase, MD 22127
				123 Magnolia Road, Hudson, N.Y. 11220

Result:

Thus the SQL commands to query the database tables and explore natural, equi and outer joins were executed successfully and their outputs were verified.

Ex. No.: 6

User Defined Functions and Stored Procedures

Date:

AIM:

To write user defined functions and stored procedures in SQL.

DESCRIPTION:

USER DEFINED FUNCTION:

- **CREATE FUNCTION:** Begins the function creation statement.
- **function_name:** Name of the function you are creating.
- **Parameters:** Optional list of parameters that the function accepts. Each parameter consists of a name and a data type (datatype). Parameters are enclosed in parentheses ().
- **RETURNS return_datatype:** Specifies the data type that the function returns. return_datatype can be any valid data type supported by your database system.
- **AS:** Indicates the beginning of the function body.
- **BEGIN ... END;** Encloses the statements that define the function's behavior.
- **Local Variables:** Optional local variables (DECLARE local_variable datatype;) that the function can use for temporary storage or calculations.
- **SQL Statements:** Actual SQL statements that perform the desired operations within the function.
- **RETURN statement:** Specifies what value or result the function should return. This can be a variable, an expression, a query result, etc.

PROCEDURE:

- **CREATE PROCEDURE:** Begins the stored procedure creation statement.
- **procedure_name:** Name of the procedure you are creating.
- **Parameters:** Optional list of parameters that the procedure can accept. Each parameter consists of a name, a data type (datatype),

and optionally a default value (default_value). Parameters are enclosed in parentheses ().

- **AS**: Indicates the beginning of the procedure body.
- **BEGIN ... END**: Encloses the statements that define the procedure's behavior.
- **SQL Statements**: Actual SQL statements that perform the desired operations within the procedure.

SYNTAX:

User Defined Functions:

```
CREATE FUNCTION function_name (  
    parameter1 datatype1,  
    parameter2 datatype2,  
    ...  
)
```

```
RETURNS return_datatype
```

```
AS
```

```
BEGIN
```

```
-- Function body with SQL statements
```

```
DECLARE local_variable1 datatype;
```

```
DECLARE local_variable2 datatype;
```

```
-- Initialization or other processing
```

```
-- SQL statements
```

```
-- RETURN statement
```

```
RETURN expression; -- expression can be a variable, calculation, query  
result, etc.
```

```
END;
```

Procedure:

```
CREATE PROCEDURE procedure_name
    [parameter1 datatype1 [= default_value1 ]]
    [, parameter2 datatype2 [= default_value2 ]]
    [...]
AS
BEGIN
    -- Procedure body with SQL statements
END;
```

PL/SQL STATEMENTS AND OUTPUTS :

```
SQL> CREATE OR REPLACE FUNCTION
GET_EVENT_COST(p_event_id NUMBER)
RETURN NUMBER
IS
    v_cost NUMBER;
BEGIN
    SELECT COST INTO v_cost
    FROM D_EVENTS
    WHERE ID = p_event_id;

    RETURN v_cost;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END;
/
```

Procedure Created or Replaced successfully

```
SQL> SELECT GET_EVENT_COST(100) AS EVENT_COST FROM  
DUAL;
```

```
EVENT_COST  
-----  
8000
```

```
SQL> SELECT GET_EVENT_COST(999) AS EVENT_COST FROM  
DUAL;
```

```
EVENT_COST  
-----
```

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_EVENT_COST(  
    p_event_id IN NUMBER,  
    p_new_cost IN NUMBER,  
    p_rows_updated OUT NUMBER  
)  
IS  
BEGIN  
    UPDATE D_EVENTS  
    SET COST = p_new_cost  
    WHERE ID = p_event_id;  
  
    p_rows_updated := SQL%ROWCOUNT;  
  
    COMMIT;  
EXCEPTION  
    WHEN OTHERS THEN  
        ROLLBACK;  
        RAISE;  
END;
```

/

Procedure Created or Replaced successfully

```
SQL> DECLARE
    v_rows_updated NUMBER;
BEGIN
    UPDATE_EVENT_COST(100, 8500, v_rows_updated);
    DBMS_OUTPUT.PUT_LINE('Rows updated: ' || v_rows_updated);
END;
/
```

Rows updated: 1

```
SQL> SELECT ID, NAME, COST FROM D_EVENTS WHERE ID = 100;
```

ID	NAME	COST
100	Peters Graduation	8500

Result:

Thus the user defined functions and stored procedures in SQL are written, executed successfully and their outputs were verified.

Ex. No.: 7

DCL and TCL Command

Date:

AIM:

To execute complex transactions and realize DCL and TCL commands.

DESCRIPTION:

DCL (Data Control Language) Commands:

a) GRANT:

- Used to give specific privileges to a user or role.
- Syntax breakdown:
 - {privileges}: Specific permissions (e.g., SELECT, INSERT, UPDATE, DELETE)

b) REVOKE:

- Used to remove previously granted privileges from a user or role.

TCL (Transaction Control Language) Commands:

a) COMMIT:

- Permanently saves all changes made during the current transaction to the database.

b) ROLLBACK:

- Undoes all changes made during the current transaction.
- Restores the database to its state at the beginning of the transaction.

SYNTAX:

DCL Commands:

Grant:

GRANT {privileges} ON {object} TO {user | role};

Revoke:

```
REVOKE {privileges} ON {object} FROM {user | role};  
-- TCL Commands
```

Commit:

```
COMMIT;
```

Rollback:

```
ROLLBACK;
```

SQL QUERY AND ITS OUTPUT :

```
SQL> CREATE USER event_manager IDENTIFIED BY password;
```

User created.

```
SQL> GRANT CREATE SESSION TO event_manager;
```

Grant succeeded.

```
SQL> GRANT SELECT, INSERT, UPDATE, DELETE ON D_EVENTS TO  
event_manager;
```

Grant succeeded.

```
SQL> CONNECT event_manager/password
```

Connected.

```
SQL> SAVEPOINT before_updates;
```

Savepoint created.

```
SQL> UPDATE D_EVENTS SET COST = COST * 1.1 WHERE COST < 9000;
```

1 rows updated.

```
SQL> UPDATE D_CLIENTS SET PHONE = 9876543210 WHERE CLIENT_NUMBER = 5922;
```

1 rows updated.

```
SQL> SELECT * FROM D_EVENTS;
```

ID	NAME	COST
100	Peters Graduation	9350
105	Vigil wedding	10000

```
SQL> SELECT * FROM D_CLIENTS WHERE CLIENT_NUMBER = 5922;
```

CLIENT_NUMBER	FIRST_NAME	LAST_NAME	PHONE	EMAIL
5922	Hiram	Peters	9876543210	hpeters@yahoo.com

```
SQL> ROLLBACK TO SAVEPOINT before_updates;
```

Rollback complete.

```
SQL> COMMIT;
```

Commit complete.

```
SQL> REVOKE DELETE ON D_EVENTS FROM event_manager;
```

Revoke succeeded.

```
SQL> DELETE FROM D_EVENTS WHERE ID = 100;  
Error: ORA-01031: insufficient privileges
```

```
SQL> CONNECT original_user/password
```

Connected.

```
SQL> DROP USER event_manager;
```

User dropped.

Result:

Thus the SQL commands to execute complex transactions and to realize DCL and TCL commands were successfully executed and their outputs were verified.

Ex. No.: 8

SQL Triggers

Date:

AIM:

To write SQL Triggers for insert, delete, and update operations in a database table.

DESCRIPTION:

Insert Trigger:

- AFTER INSERT ON table_name: Specifies that the trigger activates after an INSERT operation on the specified table.
- FOR EACH ROW: Indicates the trigger executes once for each row affected by the INSERT operation.
- BEGIN ... END: Encloses the SQL statements to be executed when the trigger fires.

Update Trigger:

- AFTER UPDATE ON table_name: Activates the trigger after an UPDATE operation on the specified table.
- Can access OLD and NEW values to compare before and after the update.

Delete Trigger:

- AFTER DELETE ON table_name: Activates the trigger after a DELETE operation on the specified table.
- Can access OLD values of the deleted row.

SYNTAX:**Insert Trigger:****After Insert:**

AFTER INSERT ON table_name

For Each Row:

FOR EACH ROW

BEGIN

-- Trigger action statements

END;

Update Trigger:

AFTER UPDATE ON table_name

FOR EACH ROW

BEGIN

-- Trigger action statements

END;

Delete Trigger:

CREATE TRIGGER trigger_name

AFTER DELETE ON table_name

FOR EACH ROW

BEGIN

-- Trigger action statements

END;

PL/SQL STATEMENTS AND ITS OUTPUT :

```
SQL> CREATE TABLE EVENT_AUDIT (  
  2  AUDIT_ID NUMBER GENERATED ALWAYS AS IDENTITY,  
  3  EVENT_ID NUMBER,  
  4  ACTION VARCHAR2(10),  
  5  ACTION_DATE DATE,  
  6  OLD_COST NUMBER,  
  7  NEW_COST NUMBER  
  8 );
```

Table created.

```
SQL> CREATE OR REPLACE TRIGGER TRG_EVENT_AUDIT  
  2 AFTER INSERT OR UPDATE OR DELETE ON D_EVENTS  
  3 FOR EACH ROW  
  4 BEGIN  
  5   IF INSERTING THEN  
  6       INSERT INTO EVENT_AUDIT (EVENT_ID, ACTION,  
ACTION_DATE, NEW_COST)  
  7       VALUES (:NEW.ID, 'INSERT', SYSDATE, :NEW.COST);  
  8   ELSIF UPDATING THEN  
  9       INSERT INTO EVENT_AUDIT (EVENT_ID, ACTION,  
ACTION_DATE, OLD_COST, NEW_COST)  
 10       VALUES (:OLD.ID, 'UPDATE', SYSDATE, :OLD.COST,  
:NEW.COST);  
 11   ELSIF DELETING THEN  
 12       INSERT INTO EVENT_AUDIT (EVENT_ID, ACTION,  
ACTION_DATE, OLD_COST)  
 13       VALUES (:OLD.ID, 'DELETE', SYSDATE, :OLD.COST);
```

```
14    END IF;  
15 END;  
16 /
```

Trigger created successfully.

```
SQL> UPDATE D_EVENTS SET COST = 8200 WHERE ID = 100;
```

1 row updated.

```
SQL> INSERT INTO D_EVENTS (ID, NAME, EVENT_DATE, COST,  
VENUE_ID, PACKAGE_CODE, THEME_CODE, CLIENT_NUMBER)  
  2 VALUES (106, 'New Event', SYSDATE, 7500, 100, 79, 200, 5922);
```

1 row inserted.

```
SQL> DELETE FROM D_EVENTS WHERE ID = 106;
```

1 row deleted.

```
SQL> SELECT * FROM EVENT_AUDIT;
```

```
AUDIT_ID  EVENT_ID ACTION  ACTION_DATE OLD_COST  
NEW_COST
```

```
-----  
  1      100 UPDATE   01-JUL-24   8500    8200  
  2      106 INSERT   01-JUL-24           7500  
  3      106 DELETE   01-JUL-24   7500
```

Result:

Thus the SQL Triggers for insert, delete, and update operations in a database table are written, executed successfully and their outputs were verified.

Ex. No.: 9

View and Index for Database Tables

Date:

AIM:

To create a view and index for database tables with a large number of records.

DESCRIPTION:

View:

A view is a virtual table based on the result of a SQL statement. It's essentially a stored query that you can refer to as if it were a table.

Index:

An index in Oracle SQL is a database object that improves the speed of data retrieval operations on database tables

SYNTAX:

View:

```
CREATE VIEW as view_name as select col1, col2... from table_name;
```

Index:

```
CREATE INDEX index_name  
ON table_name (column1, column2, ...);
```

SQL QUERY AND ITS OUTPUT :

```
SQL> CREATE INDEX IDX_EVENTS_DATE ON D_EVENTS  
(EVENT_DATE);
```

Index created.

```
SQL> SELECT INDEX_NAME, TABLE_NAME, COLUMN_NAME
2 FROM USER_IND_COLUMNS
3 WHERE TABLE_NAME = 'D_EVENTS' AND COLUMN_NAME =
'EVENT_DATE';
```

INDEX_NAME	TABLE_NAME	COLUMN_NAME
IDX_EVENTS_DATE	D_EVENTS	EVENT_DATE

```
SQL> CREATE INDEX IDX_EVENTS_YEAR ON D_EVENTS
(EXTRACT(YEAR FROM EVENT_DATE));
```

Index created.

```
SQL> SELECT * FROM D_EVENTS WHERE EXTRACT(YEAR FROM
EVENT_DATE) = 2004;
```

ID	NAME	EVENT_DATE	DESCRIPTION	COST
VENUE_ID	PACKAGE_CODE	THEME_CODE	CLIENT_NUMBER	
100	Peters Graduation	2004-05-14	Party for 200, red, white, blue...	8200
100	112	200	5922	
105	Vigil wedding	2004-04-28	Black tie at Four Season hotel	10000
220	200	200	6133	

Result:

Thus the creation of a view and index for database tables with a large number of records were executed successfully and their outputs were verified.

Ex. No: 10 XML Database and Validate it using XML Schema

Date:

AIM:

To create an XML database and validate it using XML schema.

Description:

1. XML Schema Registration:

- DBMS_XMLSCHEMA.registerSchema: Oracle Database procedure to register an XML schema.

- schemaURL: Specifies the URL identifier for the schema (e.g., 'http://example.com/schema.xsd').

- schemaDoc: Contains the actual XML schema content as a string.

- local: When TRUE, the schema is registered as a local schema.

- genTypes: When TRUE, generates object types based on the schema.

- genTables: When TRUE, generates default tables for all global elements.

- force: When FALSE, doesn't overwrite existing schemas.

- enableHierarchy: Specifies how to manage hierarchical data in generated tables.

2. XML Validation:

- l_xmltype: An XMLType variable or column containing the XML data to be validated.

- schemavalidate: Method to validate the XML data against its associated schema.

3. Displaying Registered Schemas:

- SELECT schema_url FROM user_xml_schemas: Retrieves and displays the URLs of all XML schemas registered for the current user.

- This query helps verify that the schema was successfully registered and is available for use.

SYNTAX:

Register XML Schema:

```
BEGIN
  DBMS_XMLSCHEMA.registerSchema(
    schemaURL => 'http://example.com/schema.xsd',
    schemaDoc => 'schema_content',
    local => TRUE,
    genTypes => TRUE,
    genTables => TRUE,
    force => FALSE,
    enableHierarchy =>
DBMS_XMLSCHEMA.ENABLE_ALL_HIERARCHY);
END;
```

Validate XML against Schema:

```
l_xmltype.schemavalidate;
```

Display registered XML Schemas:

```
SELECT schema_url FROM user_xml_schemas;
```

SQL QUERY AND ITS OUTPUT :

```
SQL> SELECT XMLELEMENT("Event",
2      XMLATTRIBUTES(ID AS "EventID"),
3      XMLELEMENT("Name", NAME),
4      XMLELEMENT("Date", EVENT_DATE),
5      XMLELEMENT("Cost", COST),
6      XMLELEMENT("Venue",
7      XMLATTRIBUTES(VENUE_ID AS "ID")
8      )
9      ) AS EventXML
```

```
10 FROM D_EVENTS
11 WHERE ID = 100;
```

EVENTXML

```
-----
<Event EventID="100"><Name>Peters
Graduation</Name><Date>2004-05-14</Date><Cost>8200</Cost><Venue
ID="100"/></Event>
```

```
SQL> WITH XMLData AS (
  2   SELECT XMLTYPE(
  3     '<Events>
  4       <Event EventID="100">
  5         <Name>Peters Graduation</Name>
  6         <Date>2004-05-14</Date>
  7         <Cost>8200</Cost>
  8         <Venue ID="100"/>
  9       </Event>
 10     <Event EventID="105">
 11       <Name>Vigil wedding</Name>
 12       <Date>2004-04-28</Date>
 13       <Cost>10000</Cost>
 14       <Venue ID="220"/>
 15     </Event>
 16     </Events>'
 17   ) AS xml FROM DUAL
 18 )
 19 SELECT ExtractValue(Value(e), '/Event/Name') AS EventName,
 20        ExtractValue(Value(e), '/Event/Date') AS EventDate,
 21        ExtractValue(Value(e), '/Event/Cost') AS EventCost
 22 FROM XMLData x, TABLE(XMLSEQUENCE(EXTRACT(x.xml,
'/Events/Event')))) e;
```

EVENTNAME	EVENTDATE	EVENTCOST
-----------	-----------	-----------

Peters Graduation 2004-05-14 8200
Vigil wedding 2004-04-28 10000

```
SQL> UPDATE (  
2   SELECT XMLTYPE(  
3     '<Event>  
4       <Name>Peters Graduation</Name>  
5       <Date>2004-05-14</Date>  
6       <Cost>8200</Cost>  
7     </Event>'  
8   ) AS xml FROM DUAL  
9 )  
10 SET xml = UPDATEXML(xml, '/Event/Cost/text()', '8500')  
11 WHERE ExtractValue(xml, '/Event/Name') = 'Peters Graduation';
```

1 row updated.

```
SQL> SELECT XMLTYPE(  
2   '<Event>  
3     <Name>Peters Graduation</Name>  
4     <Date>2004-05-14</Date>  
5     <Cost>8500</Cost>  
6   </Event>'  
7 ) AS UpdatedXML FROM DUAL;
```

UPDATEDXML

<Event><Name>Peters
Graduation</Name><Date>2004-05-14</Date><Cost>8500</Cost></Event
>

Result:

Thus the creation of an XML database and validating it using XML schema were successfully executed and its results were verified.

Ex. No.: 11

NOSQL Database

Date:

AIM:

To create document, column and graph based data using NOSQL database tools.

Syntax with Description;

MongoDB

MongoDB utilizes a document-based model with collections and documents.

Insert Document:

```
db.collection_name.insertOne({  
    key1: value1,  
    key2: value2,  
    // More key-value pairs  
});
```

Find Document:

```
db.collection_name.find({ key: value });
```

Cassandra

Apache Cassandra uses a column-family model with keyspaces, tables, rows, and columns.

Create Keyspace:

```
CREATE KEYSPACE keyspace_name WITH replication = {'class':  
'SimpleStrategy', 'replication_factor': 1};
```

Use the keyspace:

```
USE keyspace_name; // To
```

Create the table:

```
CREATE TABLE table_name (  
    key1 data_type PRIMARY KEY,  
    key2 data_type,  
    // More columns  
);
```

Inserts one row:

```
INSERT INTO table_name (key1, key2, ...) VALUES (value1, value2, ...);
```

Selects data:

```
SELECT * FROM table_name WHERE key1 = value;
```

Neo4j

Neo4j employs a graph-based model with nodes, relationships, and properties.

Create Node:

```
CREATE (:Label {key: value}); // Create Node
```

Creating Relationship:

```
MATCH (node1:Label1), (node2:Label2)
```

```
WHERE node1.key = value1 AND node2.key = value2  
CREATE (node1)-[:RELATIONSHIP]->(node2);
```

Querying Nodes and Relationships:

```
MATCH (node1)-[:RELATIONSHIP]->(node2)  
WHERE node1.key = value  
RETURN node1, node2;
```

QUERY:

MongoDB

Inserting document:

```
db.customers.insertOne({  
  _id: ObjectId(),  
  name: "John Doe",  
  age: 30,  
  email: "john.doe@example.com"  
});
```

Querying document:

```
db.customers.find({ name: "John Doe" });
```

OUTPUT:

Insertion

```
{  
  "_id" : ObjectId("..."),
```

```
"name" : "John Doe",  
"age" : 30,  
"email" : "john.doe@example.com"  
}
```

Query

```
{ "_id" : ObjectId("..."), "name" : "John Doe", "age" : 30, "email" :  
"john.doe@example.com" }
```

Cassandra

Creating a column-based table in Cassandra

```
CREATE KEYSPACE IF NOT EXISTS my_keyspace WITH replication =  
{'class': 'SimpleStrategy', 'replication_factor': 1};
```

USE my_keyspace;

```
CREATE TABLE IF NOT EXISTS users (  
    user_id UUID PRIMARY KEY,  
    name TEXT,  
    age INT,  
    email TEXT  
);
```

Inserting data into the column-based table

```
INSERT INTO users (user_id, name, age, email) VALUES (uuid(), 'Alice',  
28, 'alice@example.com');
```

Querying data from the column-based table

```
SELECT * FROM users WHERE name = 'Alice';
```

OUTPUT:

user_id	age	email	name
41f9c3ca-b6f2-4c6e-9e3c-6a4d601fb4d8	28	alice@example.com	Alice

Neo4j

Creating nodes and relationships in Neo4j

```
CREATE (:Person {name: 'Alice', age: 30})
CREATE (:Person {name: 'Bob', age: 25});
```

```
MATCH (alice:Person), (bob:Person)
WHERE alice.name = 'Alice' AND bob.name = 'Bob'
CREATE (alice)-[:FRIENDS]->(bob);
```

Querying data from Neo4j

```
MATCH (person:Person)-[:FRIENDS]->(friend)
WHERE person.name = 'Alice'
RETURN person, friend;
```

OUTPUT:

person	friend
(:Person {name: 'Alice'})	(:Person {name: 'Bob'})

Result:

Thus the creation of document, column and graph based data using NOSQL database were executed successfully and its outputs were verified.

.

Ex. No.: 12

Mini Project - DJ on Demand

Date:

AIM:

To build a real time database application by designing an ER diagram, applying normalization principles on the entities and incorporating the necessary SQL and PL/SQL features.

SCHEMA:

```
CREATE TABLE "D_CDS"
```

```
  ("CD_NUMBER" NUMBER(5,0),
```

```
    "TITLE" VARCHAR2(50) CONSTRAINT "D_CDS_TITLE_NN" NOT  
NULL ENABLE,
```

```
    "PRODUCER"    VARCHAR2(50)    CONSTRAINT  
"D_CDS_PRODUCER_NN" NOT NULL ENABLE,
```

```
    "YEAR" VARCHAR2(4) CONSTRAINT "D_CDS_YEAR_NN" NOT NULL  
ENABLE,
```

```
    CONSTRAINT "D_CDS_CD_NUMBER_PK" PRIMARY KEY  
("CD_NUMBER")
```

```
  USING INDEX ENABLE
```

```
);
```

```
CREATE TABLE "D_CLIENTS"
```

```
  ("CLIENT_NUMBER" NUMBER(5,0),
```

```
    "FIRST_NAME"    VARCHAR2(25)    CONSTRAINT  
"D_CLT_FIST_NAME_NN" NOT NULL ENABLE,
```

```
        "LAST_NAME"    VARCHAR2(30)    CONSTRAINT
"D_CLT_LAST_NAME_NN" NOT NULL ENABLE,
        "PHONE" NUMBER(15,0) CONSTRAINT "D_CLT_PHONE_NN" NOT
NULL ENABLE,
        "EMAIL" VARCHAR2(50),
        CONSTRAINT "D_CLT_CLIENT_NUMBER_PK" PRIMARY KEY
("CLIENT_NUMBER")
        USING INDEX ENABLE
    );
```

```
CREATE TABLE "D_PACKAGES"
("CODE" NUMBER(10,0),
        "LOW_RANGE"    NUMBER(6,0)    CONSTRAINT
"D_PKE_LOW_RANGE_NN" NOT NULL ENABLE,
        "HIGH_RANGE"    NUMBER(6,0)    CONSTRAINT
"D_PKE_HIGH_RANGE_NN" NOT NULL ENABLE,
        CONSTRAINT "D_PKE_CODE_PK" PRIMARY KEY ("CODE")
        USING INDEX ENABLE
    );
```

```
CREATE TABLE "D_THEMES"
("CODE" NUMBER(10,0),
        "DESCRIPTION" VARCHAR2(100) CONSTRAINT "D_TME_DESC_NN"
NOT NULL ENABLE,
        CONSTRAINT "D_TME_CODE_PK" PRIMARY KEY ("CODE")
        USING INDEX ENABLE
    );
```

```

CREATE TABLE "D_VENUES"
  ("ID" NUMBER(5,0),
   "LOC_TYPE" VARCHAR2(30) CONSTRAINT "D_VNE_LOC_TYPE_NN"
NOT NULL ENABLE,
   "ADDRESS" VARCHAR2(100) CONSTRAINT "D_VNE_ADDRESS_NN"
NOT NULL ENABLE,
   "RENTAL_FEE"    VARCHAR2(50)    CONSTRAINT
"D_VNE_RENTAL_FEE_NN" NOT NULL ENABLE,
   "COMMENTS" VARCHAR2(100),
   CONSTRAINT "D_VNE_ID_PK" PRIMARY KEY ("ID")
USING INDEX ENABLE
);

```

```

CREATE TABLE "D_EVENTS"
  ("ID" NUMBER(5,0),
   "NAME" VARCHAR2(50) CONSTRAINT "D_EVE_NAME_NN" NOT
NULL ENABLE,
   "EVENT_DATE" DATE CONSTRAINT "D_EVE_EVENT_DATE_NN"
NOT NULL ENABLE,
   "DESCRIPTION" VARCHAR2(50),
   "COST" NUMBER(8,2) CONSTRAINT "D_EVE_COST_NN" NOT NULL
ENABLE,
   "VENUE_ID" NUMBER(5,0) CONSTRAINT "D_EVE_VENUE_ID_NN"
NOT NULL ENABLE,
   "PACKAGE_CODE"    NUMBER(5,0)    CONSTRAINT
"D_EVE_PACKAGE_CODE_NN" NOT NULL ENABLE,

```

```

            "THEME_CODE"    NUMBER(10,0)    CONSTRAINT
"D_EVE_THEME_CODE_NN" NOT NULL ENABLE,
            "CLIENT_NUMBER"    NUMBER(5,0)    CONSTRAINT
"D_EVE_CLIENT_NUMBER_NN" NOT NULL ENABLE,
            CONSTRAINT "D_EVE_ID_PK" PRIMARY KEY ("ID")
            USING INDEX ENABLE
        );

```

```

CREATE TABLE "D_PARTNERS"
    ("ID" NUMBER(5,0),
            "FIRST_NAME"    VARCHAR2(25)    CONSTRAINT
"D_PTR_FIRST_NAME_NN" NOT NULL ENABLE,
            "LAST_NAME"    VARCHAR2(30)    CONSTRAINT
"D_PTR_LAST_NAME_NN" NOT NULL ENABLE,
            "EXPERTISE" VARCHAR2(25),
            "SPECIALTY" VARCHAR2(25),
            "AUTH_EXPENSE_AMT" NUMBER(8,2),
            "MANAGER_ID" NUMBER(5,0),
            "PARTNER_TYPE"    VARCHAR2(25)    CONSTRAINT
"D_PTR_PARTNER_TYPE_NN" NOT NULL ENABLE,
            CONSTRAINT "D_PTR_ID_PK" PRIMARY KEY ("ID")
            USING INDEX ENABLE
    );

```

```

CREATE TABLE "D_JOB_ASSIGNMENTS"
    ("PARTNER_ID" NUMBER(5,0),
        "EVENT_ID" NUMBER(5,0),

```

```
"JOB_DATE" DATE CONSTRAINT "D_JAT_JOB_DATE_NN" NOT NULL
ENABLE,
"STATUS" VARCHAR2(50),
CONSTRAINT "D_JAT_PK" PRIMARY KEY ("PARTNER_ID",
"EVENT_ID")
USING INDEX ENABLE
);
```

```
CREATE TABLE "D_TYPES"
("CODE" NUMBER(10,0),
"DESCRIPTION" VARCHAR2(50) CONSTRAINT "D_TPE_DESC_NN"
NOT NULL ENABLE,
CONSTRAINT "D_TPE_CODE_PK" PRIMARY KEY ("CODE")
USING INDEX ENABLE
);
```

```
CREATE TABLE "D_SONGS"
("ID" NUMBER(5,0),
"TITLE" VARCHAR2(50) CONSTRAINT "D_SNG_TITLE_NN" NOT
NULL ENABLE,
"DURATION" VARCHAR2(20),
"ARTIST" VARCHAR2(20),
"TYPE_CODE" NUMBER(5,0) CONSTRAINT
"D_SNG_TYPE_CODE_NN" NOT NULL ENABLE,
CONSTRAINT "D_SNG_ID_PK" PRIMARY KEY ("ID")
USING INDEX ENABLE
);
```

```

CREATE TABLE "D_PLAY_LIST_ITEMS"
("EVENT_ID" NUMBER(5,0),
"SONG_ID" NUMBER(5,0),
"COMMENTS" VARCHAR2(80),
CONSTRAINT "D_PLM_PK" PRIMARY KEY ("EVENT_ID",
"SONG_ID")
USING INDEX ENABLE
);

```

```

CREATE TABLE "D_TRACK_LISTINGS"
("SONG_ID" NUMBER(5,0),
"CD_NUMBER" NUMBER(5,0),
"TRACK" NUMBER(2,0) CONSTRAINT
"D_TLG_TRACK_NUMBER_NN" NOT NULL ENABLE,
CONSTRAINT "D_TLG_PK" PRIMARY KEY ("SONG_ID",
"CD_NUMBER")
USING INDEX ENABLE
);

```

```

ALTER TABLE "D_EVENTS" ADD CONSTRAINT
"D_EVE_CLIENT_NUMBER_FK" FOREIGN KEY ("CLIENT_NUMBER")
REFERENCES "D_CLIENTS" ("CLIENT_NUMBER") ENABLE;
ALTER TABLE "D_EVENTS" ADD CONSTRAINT
"D_EVE_PACKAGE_CODE_FK" FOREIGN KEY ("PACKAGE_CODE")
REFERENCES "D_PACKAGES" ("CODE") ENABLE;

```

```
ALTER TABLE "D_EVENTS" ADD CONSTRAINT  
"D_EVE_THEME_CODE_FK" FOREIGN KEY ("THEME_CODE")
```

```
REFERENCES "D_THEMES" ("CODE") ENABLE;
```

```
ALTER TABLE "D_EVENTS" ADD CONSTRAINT  
"D_EVE_VENUE_ID_FK" FOREIGN KEY ("VENUE_ID")
```

```
REFERENCES "D_VENUES" ("ID") ENABLE;
```

```
ALTER TABLE "D_JOB_ASSIGNMENTS" ADD CONSTRAINT  
"D_JAT_EVENT_ID_FK" FOREIGN KEY ("EVENT_ID")
```

```
REFERENCES "D_EVENTS" ("ID") ENABLE;
```

```
ALTER TABLE "D_JOB_ASSIGNMENTS" ADD CONSTRAINT  
"D_JAT_PARTNER_ID_FK" FOREIGN KEY ("PARTNER_ID")
```

```
REFERENCES "D_PARTNERS" ("ID") ENABLE;
```

```
ALTER TABLE "D_PLAY_LIST_ITEMS" ADD CONSTRAINT  
"D_PLM_EVENT_ID_FK" FOREIGN KEY ("EVENT_ID")
```

```
REFERENCES "D_EVENTS" ("ID") ENABLE;
```

```
ALTER TABLE "D_PLAY_LIST_ITEMS" ADD CONSTRAINT  
"D_PLM_SONG_ID_FK" FOREIGN KEY ("SONG_ID")
```

```
REFERENCES "D_SONGS" ("ID") ENABLE;
```

```
ALTER TABLE "D_SONGS" ADD CONSTRAINT  
"D_SNG_TYPE_CODE_FK" FOREIGN KEY ("TYPE_CODE")
```

```
REFERENCES "D_TYPES" ("CODE") ENABLE;
```

```
ALTER TABLE "D_TRACK_LISTINGS" ADD CONSTRAINT  
"D_TLG_CD_NUMBER_FK" FOREIGN KEY ("CD_NUMBER")
```

```
REFERENCES "D_CDS" ("CD_NUMBER") ENABLE;
```

```
ALTER TABLE "D_TRACK_LISTINGS" ADD CONSTRAINT  
"D_TLG_SONG_ID_FK" FOREIGN KEY ("SONG_ID")
```

```
REFERENCES "D_SONGS" ("ID") ENABLE;
```

```
CREATE INDEX "D_CDS_IDX" ON "D_CDS" ("TITLE");  
CREATE INDEX "SONG_ID_IDX" ON "D_TRACK_LISTINGS"  
("SONG_ID");
```

```
-- populate tables
```

```
INSERT INTO d_cds(cd_number,title,producer,year)  
VALUES(90,'The Celebrants Live in Concert','Old Town Records','1997');  
INSERT INTO d_cds(cd_number,title,producer,year)  
VALUES(91,'Party Music for All Occasions','The Music Man','2000');  
INSERT INTO d_cds(cd_number,title,producer,year)  
VALUES(92,'Back to the Shire','Middle Earth Records','2002');  
INSERT INTO d_cds(cd_number,title,producer,year)  
VALUES(93,'Songs from My Childhood','Old Town Records','1999');  
INSERT INTO d_cds(cd_number,title,producer,year)  
VALUES(94,'Carpe Diem','R & B Inc.','2000');  
INSERT INTO d_cds(cd_number,title,producer,year)  
VALUES(95,'Here Comes the Bride','The Music Man','2001');  
INSERT INTO d_cds(cd_number,title,producer,year)  
VALUES(96,'Graduation Songbook','Tunes Are Us','1998');  
INSERT INTO d_cds(cd_number,title,producer,year)  
VALUES(98,'Whirled Peas','Old Town Records','2004');
```

```
INSERT INTO d_clients(client_number,first_name,last_name,phone,email)  
VALUES(5922,'Hiram','Peters',3715832249,'hpeters@yahoo.com');  
INSERT INTO d_clients(client_number,first_name,last_name,phone,email)  
VALUES(5857,'Serena','Jones',7035335900,'serena.jones@jones.com');
```

```
INSERT INTO d_clients(client_number,first_name,last_name,phone,email)
VALUES(6133,'Lauren','Vigil',4072220090,'lbv@lbv.net');
```

```
INSERT INTO d_packages(code,low_range,high_range)
VALUES(79,500,2500);
```

```
INSERT INTO d_packages(code,low_range,high_range)
VALUES(87,2501,5000);
```

```
INSERT INTO d_packages(code,low_range,high_range)
VALUES(112,5001,10000);
```

```
INSERT INTO d_packages(code,low_range,high_range)
VALUES(200,10001,15000);
```

```
INSERT INTO d_themes(code,description)
VALUES(200,'Tropical');
```

```
INSERT INTO d_themes(code,description)
VALUES(220,'Carnival');
```

```
INSERT INTO d_themes(code,description)
VALUES(240,'Sixties');
```

```
INSERT INTO d_themes(code,description)
VALUES(110,'Classic');
```

```
INSERT INTO d_themes(code,description)
VALUES(198,'Mardi Gras');
```

```
INSERT INTO d_themes(code,description)
VALUES(454,'Eighties');
```

```
INSERT INTO d_themes(code,description)
```

```
VALUES(340,'Football');
```

```
INSERT INTO d_themes(code,description)
```

```
VALUES(502,'Fairy Tale');
```

```
INSERT INTO d_venues(id,loc_type,address,rental_fee,comments)
```

```
VALUES(100,'Private Home','52 West End Drive, Los Angeles, CA  
90210','0','Large kitchen, spacious lawn');
```

```
INSERT INTO d_venues(id,loc_type,address,rental_fee,comments)
```

```
VALUES(105,'Private Home','123 Magnolia Road, Hudson, N.Y.  
11220','0','3 level townhouse, speakers on all floors');
```

```
INSERT INTO d_venues(id,loc_type,address,rental_fee,comments)
```

```
VALUES(101,'Private Home','4 Primrose Lane, Chevy Chase, MD  
22127','0','Gazebo, multi-level deck');
```

```
INSERT INTO d_venues(id,loc_type,address,rental_fee,comments)
```

```
VALUES(95,'School Hall','4 Mahogany Drive, Boston, MA  
10010','75/hour','School closes at 10pm');
```

```
INSERT INTO d_venues(id,loc_type,address,rental_fee,comments)
```

```
VALUES(99,'National Park','87 Park Avenue, San Diego, CA  
28978','400/flat fee','Bring generators');
```

```
INSERT INTO d_venues(id,loc_type,address,rental_fee,comments)
```

```
VALUES(220,'Hotel','200 Pennsylvania Ave, Washington D.C.  
09002','300/per person','Classy affair, tight security, private entrance for  
vendors');
```

```
INSERT INTO  
d_events(client_number,id,name,event_date,description,cost,venue_id,pac  
kage_code,theme_code)
```

```
VALUES(5922,100,'Peters  
Graduation',TO_DATE('05-14-2004','mm-dd-yyyy'),'Party for 200, red,  
white, blue motif',8000,100,112,200);
```

```
INSERT INTO  
d_events(client_number,id,name,event_date,description,cost,venue_id,pac  
kage_code,theme_code)
```

```
VALUES(6133,105,'Vigil  
wedding',TO_DATE('04-28-2004','mm-dd-yyyy'),'Black tie at Four Season  
hotel',10000,220,200,200);
```

```
INSERT INTO  
d_partners(id,first_name,last_name,expertise,specialty,auth_expense_amt,  
manager_id,partner_type)
```

```
VALUES(11,'Jennifer','cho','Weddings','All Types',NULL,33,'Wedding  
Coordinator');
```

```
INSERT INTO  
d_partners(id,first_name,last_name,expertise,specialty,auth_expense_amt,  
manager_id,partner_type)
```

```
VALUES(22,'Jason','Tsang',NULL,'Hip Hop',NULL,33,'Disk Jockey');
```

```
INSERT INTO  
d_partners(id,first_name,last_name,expertise,specialty,auth_expense_amt,  
manager_id,partner_type)
```

```
VALUES(33,'Allison','Plumb','Event Planning',NULL,300000,33,'Manager');
```

```
INSERT INTO d_job_assignments(partner_id,event_id,job_date,status)
```

```
VALUES(11,105,TO_DATE('02-02-2004','mm-dd-yyyy'),'Visited');
```

```
INSERT INTO d_types(code,description)
```

```
VALUES(1,'Jazz');
```

```
INSERT INTO d_types(code,description)
```

```
VALUES(12,'Pop');
INSERT INTO d_types(code,description)
VALUES(40,'Reggae');
INSERT INTO d_types(code,description)
VALUES(88,'Country');
INSERT INTO d_types(code,description)
VALUES(77,'New Age');
```

```
INSERT INTO d_songs(id,title,duration,artist,type_code)
VALUES(45,'Its Finally Over','5 min','The Hobbits',12);
INSERT INTO d_songs(id,title,duration,artist,type_code)
VALUES(46,'Im Going to Miss My Teacher','2 min','Jane Pop',12);
INSERT INTO d_songs(id,title,duration,artist,type_code)
VALUES(47,'Hurrah for Today','3 min','The Jubilant Trio',77);
INSERT INTO d_songs(id,title,duration,artist,type_code)
VALUES(48,'Meet Me At the Altar','6 min','Bobby West',1);
INSERT INTO d_songs(id,title,duration,artist,type_code)
VALUES(49,'Lets Celebrate','8 min','The Celebrants',77);
INSERT INTO d_songs(id,title,duration,artist,type_code)
VALUES(50,'All These Years','10 min','Diana Crooner',88);
```

```
INSERT INTO d_play_list_items(event_id,song_id,comments)
VALUES(100,45,'Play late');
INSERT INTO d_play_list_items(event_id,song_id,comments)
VALUES(100,46,NULL);
```

```
INSERT INTO d_play_list_items(event_id,song_id,comments)
VALUES(100,47,'Play early');
INSERT INTO d_play_list_items(event_id,song_id,comments)
VALUES(105,48,'Play after cake cutting');
INSERT INTO d_play_list_items(event_id,song_id,comments)
VALUES(105,49,'Play first');
INSERT INTO d_play_list_items(event_id,song_id,comments)
VALUES(105,47,'Play for the father');
```

```
INSERT INTO d_track_listings(song_id,cd_number,track)
VALUES(45,92,1);
INSERT INTO d_track_listings(song_id,cd_number,track)
VALUES(46,93,1);
INSERT INTO d_track_listings(song_id,cd_number,track)
VALUES(47,91,2);
INSERT INTO d_track_listings(song_id,cd_number,track)
VALUES(48,95,5);
INSERT INTO d_track_listings(song_id,cd_number,track)
VALUES(49,91,3);
```

DESCRIPTION:

1. Import Statements:

- Flask: Web framework for creating the application.
- render_template, request, etc.: Flask modules for handling web requests and responses.

- dbConnect: Custom module for database operations.

2. Application Setup:

- app = Flask(__name__): Initializes the Flask application.
- app.secret_key: Sets a secret key for session management.

3. Route Definitions:

- @app.route('/'): Defines the home page route.
- @app.route('/add_entity', methods=['POST']): Handles entity addition.
 - @app.route('/update_entity', methods=['POST']): Handles entity updates.
- @app.route('/delete_entity', methods=['POST']): Handles entity deletion.
- @app.route('/view_entity', methods=['POST']): Handles entity viewing.

4. Database Operations:

- dbConnect.read_db_config(): Reads database configuration.
- dbConnect.execute_sql(): Executes SQL queries.

5. Request Handling:

- request.form[]: Retrieves form data submitted by the user.

6. Response Rendering:

- render_template(): Renders HTML templates with data.
- flash(): Provides feedback messages to the user.

7. Error Handling:

- try-except blocks: Handle potential errors in database operations.

8. Main Execution:

- if `__name__ == '__main__':` Runs the Flask application in debug mode.

SYNTAX:

```
python
```

```
from flask import Flask, render_template, request, redirect, url_for, flash
import dbConnect
```

```
app = Flask(__name__)
app.secret_key = 'supersecretkey'
```

```
@app.route('/')
def home():
    return render_template('index.html')
```

```
@app.route('/add_entity', methods=['POST'])
def add_entity():
    # Entity addition logic
    return render_template('success.html')
```

```
@app.route('/update_entity', methods=['POST'])
def update_entity():
    # Entity update logic
```

```
    return render_template('success.html')
```

```
@app.route('/delete_entity', methods=['POST'])
```

```
def delete_entity():
```

```
    # Entity deletion logic
```

```
    return render_template('success.html')
```

```
@app.route('/view_entity', methods=['POST'])
```

```
def view_entity():
```

```
    # Entity viewing logic
```

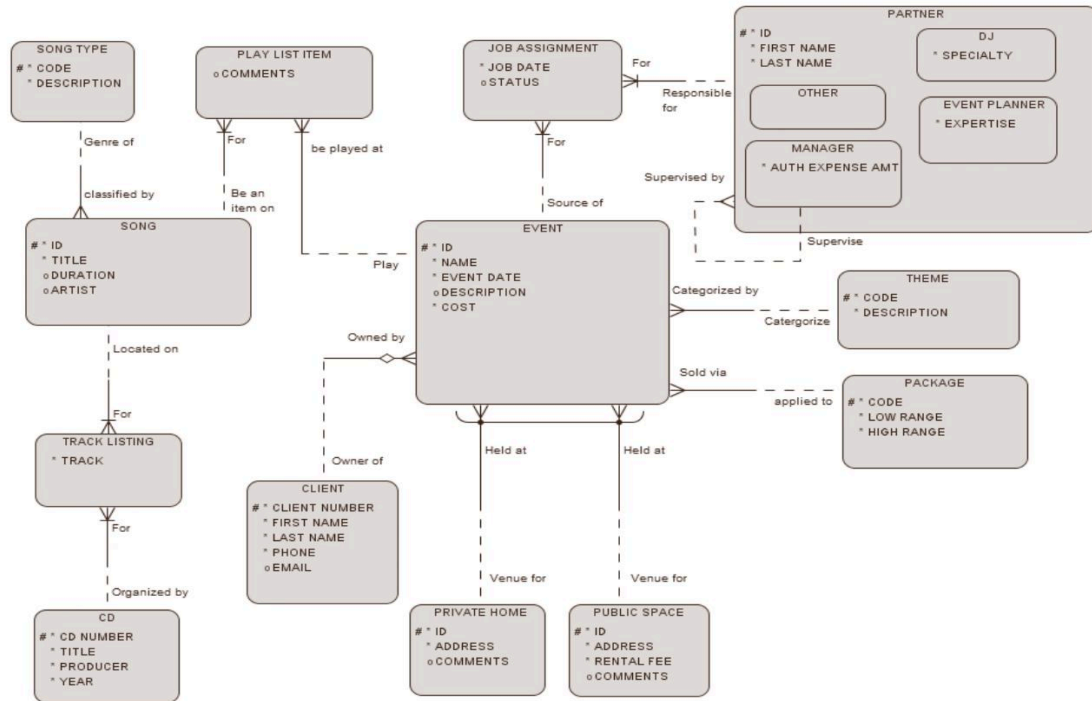
```
    return render_template('view.html', data=data)
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

ER DIAGRAM:

DJs on Demand ERD



PROGRAM:

app.py

```
from flask import Flask, render_template, request, redirect, url_for, flash
import dbConnect
```

```
app = Flask(__name__)
```

```
app.secret_key = 'supersecretkey'
```

```
@app.route('/')
```

```
def home():
```

```
    return render_template('index.html')
```

```

@app.route('/add_Packages', methods=['POST'])
def add_Packages():
    Packages_name = str(request.form['PackagesName'])
    Low_Range = str(request.form['Low_Range'])
    High_Range = str(request.form['High_Range'])
    query="insert into D_PACKAGES(code,low_range,high_range)
values("+Packages_name+","+Low_Range+","+High_Range+)"
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    return render_template('sucess.html')

@app.route('/update_Packages', methods=['POST'])
def update_Packages():
    Packages_name = str(request.form['PackagesName'])
    Low_Range = str(request.form['Low_Range'])
    High_Range = str(request.form['High_Range'])
    query="update D_PACKAGES set low_range ="+Low_Range+", high_range =
"+High_Range+" where code = "+Packages_name
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    print(query)
    return render_template('sucess.html')

@app.route('/delete_Packages', methods=['POST'])
def delete_Packages():
    Packages_name = str(request.form['PackagesName'])
    query="DELETE FROM D_PACKAGES WHERE CODE = "+Packages_name
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)

```

```
print(query)
return render_template('sucess.html')
```

```
@app.route('/view_Packages', methods=['POST'])
def view_Packages():
    Packages_name = request.form['PackagesName']
    query="select * from D_PACKAGES where code="+Packages_name
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    try:
        id, low, high = result[0]
        data = {
            "code": f"{id}",
            "low": f"{low}",
            "high": str(high)
        }
        return render_template('viewpack.html', data=data)

    except:
        return render_template('404.html')
```

Client Management Routes

```
@app.route('/add_client', methods=['POST'])
def add_client():
    client_id=str(request.form["clientId"])
    client_name = request.form['clientName']
    client_email = request.form['clientEmail']
```

```

client_phone =str( request.form['clientPhone'])
flash('Client added successfully!')

query="INSERT INTO d_clients(client_number,first_name,last_name,phone,email)
VALUES(
"+client_id+", '"+client_name+"', '"+client_name+'", "+client_phone+"', '"+client_email+'")"
print(query)
db_config = dbConnect.read_db_config()
result=dbConnect.execute_sql(db_config,query)

return render_template('sucess.html')
@app.route('/update_client', methods=['POST'])
def update_client():
    client_id = str(request.form['clientId'])
    name =request.form['clientName'],
    email= request.form['clientEmail'],
    phone= str(request.form['clientPhone'])
    print(client_id,name,phone,email)

    query="UPDATE d_clients SET first_name = '"+name[0]+'", phone= "+phone+", email
= '"+email[0]+' WHERE client_number = "+client_id
    print(query)
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    print("done")
    flash("NOT UPDATED")
    print("except")
    return render_template('sucess.html')
@app.route('/delete_client', methods=['POST'])
def delete_client():
    print("in")

```

```

client_id = str(request.form['clientId'])
if 1==1:
    print("inin")
    query="DELETE FROM d_clients WHERE client_number = "+client_id
    print("ininin")
    db_config = dbConnect.read_db_config()
    print("inininin")
    result=dbConnect.execute_sql(db_config,query)
    print("done")
    flash('Client deleted successfully!')
else:
    flash('Client not found!')

return render_template('sucess.html')
@app.route('/view_client', methods=['POST'])
def view_client():
    client_id = str(request.form['clientId'])
    query="select * from d_clients where client_number="+client_id
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    print(result)
    try:
        id, first_name, last_name, phone, email = result[0]
        data = {
            "name": f"{first_name} {last_name}",
            "phone": phone,
            "id": str(id),
            "email": email
        }

```

```

        return render_template('view.html', data=data)
    except:
        return render_template('404.html')
# Event Management Routes
@app.route('/create_event', methods=['POST'])
def create_event():
    event_id = str(request.form['eventId'])
    event_name = request.form['eventName']
    event_date = str(request.form['eventDate'])
    event_desc = request.form['eventLocation']
    event_cost = str(request.form['assignedDj'])
    venue=str(request.form['eventVenue'])
    PACKAGE_CODE=str(request.form['PACKAGE_CODE'])
    THEME_CODE=str(request.form['THEME_CODE'])
    CLIENT_NUMBER=str(request.form['CLIENT_NUMBER'])
    query="INSERT INTO
d_events(client_number,id,name,event_date,description,cost,venue_id,package_code,t
heme_code)
VALUES("+CLIENT_NUMBER+","+event_id+","+event_name+",TO_DATE('04-28-2004
','mm-dd-yyyy'),"+event_desc+","+event_cost+","+venue+","+PACKAGE_CODE+","+T
HEME_CODE+")"
    print(query)
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    flash('Event created successfully!')
    return render_template('sucess.html')
@app.route('/update_event', methods=['POST'])
def update_event():
    event_id = str(request.form['eventId'])
    event_name = request.form['eventName']

```

```

event_date = str(request.form['eventDate'])
event_desc = request.form['eventLocation']
event_cost = str(request.form['assignedDj'])
venue=str(request.form['eventVenue'])
PACKAGE_CODE=str(request.form['PACKAGE_CODE'])
THEME_CODE=str(request.form['THEME_CODE'])
CLIENT_NUMBER=str(request.form['CLIENT_NUMBER'])

query="UPDATE d_events set CLIENT_NUMBER = "+CLIENT_NUMBER+",NAME =
"+event_name+",EVENT_DATE
=
TO_DATE('04-28-2004','mm-dd-yyyy'),DESCRIPTION = "+event_desc+",COST =
"+event_cost+",VENUE_ID = "+venue+",PACKAGE_CODE = "+PACKAGE_CODE+",
THEME_CODE = "+THEME_CODE+"WHERE ID = "+event_id

print(query)

db_config = dbConnect.read_db_config()
result=dbConnect.execute_sql(db_config,query)
return render_template('sucess.html')

@app.route('/cancel_event', methods=['POST'])
def cancel_event():
    event_id = str(request.form['eventId'])
    query="DELETE FROM d_events WHERE ID = "+event_id
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    return render_template('sucess.html')

@app.route('/view_event', methods=['POST'])
def view_event():
    event_id = str(request.form['eventId'])
    query="select * from d_events where code="+event_id
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    try:

```

```

id, name, date,desc,cost,venue,pack,theme,client = result[0]

data = {
    "id": f"{id}",
    "name": f"{name}",
    "date": str(date),
    "desc": f"{desc}",
    "cost": f"{cost}",
    "venue": f"{venue}",
    "pack": f"{pack}",
    "theme": f"{theme}",
    "client": f"{client}"
}

return render_template('viewEvent.html', data=data)

```

except:

```

    return render_template('404.html')
return redirect(url_for('home'))

```

Booking Management Routes

```
@app.route('/new_booking', methods=['POST'])
```

```
def new_booking():
```

```
    Theme_desc = str(request.form['bookingClient'])
```

```
    Theme_id = str(request.form['bookingId'])
```

```

                                query="INSERT          INTO          d_themes(code,description)
VALUES("+Theme_id+", '"+Theme_desc+"")"

```

```
    db_config = dbConnect.read_db_config()
```

```
    result=dbConnect.execute_sql(db_config,query)
```

```
    return render_template('sucess.html')
```

```
@app.route('/update_booking', methods=['POST'])
```

```

def update_booking():
    Theme_desc = str(request.form['bookingClient'])
    Theme_id = str(request.form['bookingId'])
    query="UPDATE D_THEMES SET DESCRIPTION = '"+Theme_desc+"' WHERE
CODE = '"+Theme_id
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    return render_template('sucess.html')
@app.route('/cancel_booking', methods=['POST'])
def cancel_booking():
    Theme_id = str(request.form['bookingId'])
    query="DELETE FROM D_THEMES WHERE CODE = '"+Theme_id
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    return render_template('sucess.html')
@app.route('/view_booking', methods=['POST'])
def view_booking():
    theme_id = str(request.form['bookingId'])
    query="select * from D_THEMES where code='"+theme_id
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)

    try:
        code, desc = result[0]
        data = {
            "code": f"{id}",
            "desc": f"{desc}"
        }
    return render_template('viewTheme.html', data=data)

```

except:

```
    return render_template('404.html')
```

```
    return redirect(url_for('home'))
```

Payment Management Routes

```
@app.route('/add_CD', methods=['POST'])
```

```
def add_CD():
```

```
    id = request.form['CDId']
```

```
    title = request.form['CDClient']
```

```
    year = request.form['CDAmount']
```

```
        query="INSERT INTO d_cds(cd_number,title,producer,year)
VALUES("+id+", '"+title+"', 'Old Town Records', "+year+)"
```

```
    db_config = dbConnect.read_db_config()
```

```
    result=dbConnect.execute_sql(db_config,query)
```

```
    flash('Payment added successfully!')
```

```
    return render_template('sucess.html')
```

```
@app.route('/update_CD', methods=['POST'])
```

```
def update_CD():
```

```
    id = request.form['CDId']
```

```
    title = request.form['CDClient']
```

```
    year = request.form['CDAmount']
```

```
        query="UPDATE D_CDS SET TITLE = '"+title+"',year = "+year+" WHERE
CD_NUMBER = "+id
```

```
    db_config = dbConnect.read_db_config()
```

```
    result=dbConnect.execute_sql(db_config,query)
```

```
    return render_template('sucess.html')
```

```
@app.route('/delete_CD', methods=['POST'])
```

```
def delete_CD():
```

```

id = request.form['CDId']
query="DELETE FROM D_CDS WHERE CD_NUMBER = "+id
db_config = dbConnect.read_db_config()
result=dbConnect.execute_sql(db_config,query)
return render_template('sucess.html')
@app.route('/view_CD', methods=['POST'])
def view_CD():
    id = request.form['CDId']
    query="select * from D_CDS where code="+id
    db_config = dbConnect.read_db_config()
    result=dbConnect.execute_sql(db_config,query)
    try:
        id, title, producer,year = result[0]
        data = {
            "id": f"{id}",
            "year": f"{year}",
            "title":f"{title}"
        }
        return render_template('viewCd.html', data=data)
    except:
        return render_template('404.html')
    return redirect(url_for('home'))

```

```

if __name__ == '__main__':
    app.run(debug=True)

```

dbConnect.py

```

import cx_Oracle
import configparser

```

```

def read_db_config():
    config = configparser.ConfigParser()
    config.read('db_config.ini')
    if 'oracle' not in config:
        print("The 'oracle' section is missing in the configuration file.")
        return None
    db_config = {}
    try:
        db_config['username'] = config['oracle']['username']
        db_config['password'] = config['oracle']['password']
        db_config['dsn'] = config['oracle']['dsn']
    except KeyError as e:
        print(f"Missing key in configuration file: {e}")
        return None
    return db_config

def execute_sql(db_config, sql):
    try:
        connection = cx_Oracle.connect(user=db_config['username'],
        password=db_config['password'], dsn=db_config['dsn'])
        cursor = connection.cursor()
        cursor.execute(sql)

        if sql.strip().lower().startswith("select"):
            result=[]
            result = cursor.fetchall()
            if result:
                for row in result:

```

```

        print(row)
    else:
        print("No results found.")
    else:
        connection.commit()
        print("SQL statement executed successfully.")

    cursor.close()
    connection.close()
    return result
except cx_Oracle.DatabaseError as e:
    print(f"Error executing SQL statement: {e}")

if __name__ == "__main__":
    db_config = read_db_config()
    if db_config:
        # Example usage:
        execute_sql(db_config, "SELECT * FROM studentmaster") # Select query
        execute_sql(db_config, "INSERT INTO studentmaster (id, name) VALUES (1, 'John Doe')") # Insert query
        execute_sql(db_config, "UPDATE studentmaster SET name='Jane Doe' WHERE id=1") # Update query
        execute_sql(db_config, "DELETE FROM studentmaster WHERE id=1") # Delete query
    else:
        print("Failed to read database configuration.")

```

index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DJ's on Demand</title>
  <link rel="stylesheet" href="static/style.css">
</head>
<body>
  <h1>DJ'S ON DEMAND</h1>
  <div class="container">
    <div class="tabs">
      <button class="tablinks" onclick="openTab(event, 'PackagesManagement')"
id="defaultOpen">Packages Management</button>
      <button class="tablinks" onclick="openTab(event, 'ClientManagement')">Client
Management</button>
      <button class="tablinks" onclick="openTab(event, 'EventManager')">Event
Management</button>
      <button class="tablinks" onclick="openTab(event,
'BookingManagement')">Theme Management</button>
      <button class="tablinks" onclick="openTab(event, 'CDManagement')">CD
Management</button>
    </div>

    <!-- Packages Management Tab -->
    <div id="PackagesManagement" class="tabcontent">
      <h3>Packages Management</h3>
      <button onclick="openModal('addPackagesModal')">Add Packages</button>

```

```
        <button onclick="openModal('updatePackagesModal')">Update Packages
Information</button>
        <button onclick="openModal('deletePackagesModal')">Delete
Packages</button>
        <button onclick="openModal('viewPackagesModal')">View Packages
Profile</button>
    </div>
```

```
<!-- Client Management Tab -->
<div id="ClientManagement" class="tabcontent">
    <h3>Client Management</h3>
    <button onclick="openModal('addClientModal')">Add Client</button>
        <button onclick="openModal('updateClientModal')">Update Client
Information</button>
        <button onclick="openModal('deleteClientModal')">Delete Client</button>
        <button onclick="openModal('viewClientModal')">View Client Profile</button>
</div>
```

```
<!-- Event Management Tab -->
<div id="EventManagerment" class="tabcontent">
    <h3>Event Management</h3>
    <button onclick="openModal('createEventModal')">Create Event</button>
    <button onclick="openModal('updateEventModal')">Update Event</button>
    <button onclick="openModal('cancelEventModal')">Cancel Event</button>
    <button onclick="openModal('viewEventModal')">View Event Details</button>
</div>
```

```
<!-- Booking Management Tab -->
<div id="BookingManagement" class="tabcontent">
    <h3>Booking Management</h3>
```

```

        <button onclick="openModal('newBookingModal')">New Theme</button>
        <button onclick="openModal('updateBookingModal')">Update Theme</button>
        <button onclick="openModal('cancelBookingModal')">Delete Theme</button>
        <button onclick="openModal('viewBookingModal')">View Theme
Details</button>
    </div>

```

```

<!-- CD Management Tab -->
<div id="CDManagement" class="tabcontent">
    <h3>CD Management</h3>
    <button onclick="openModal('addCDModal')">Add CD</button>
    <button onclick="openModal('updateCDModal')">Update CD</button>
    <button onclick="openModal('deleteCDModal')">Delete CD</button>
    <button onclick="openModal('viewCDModal')">View CD History</button>
</div>
</div>

```

```

<!-- Modals for Packages Management -->
<div id="addPackagesModal" class="modal">
    <div class="modal-content">
        <span class="close"
onclick="closeModal('addPackagesModal')">&times;</span>
        <h2>Add Packages</h2>
        <form action="/add_Packages" method="POST">
            <label for="PackagesName">Code:</label>
            <input type="number" id="PackagesName" name="PackagesName"><br>
            <label for="Low_Range">Low_Range:</label>
            <input type="number" id="Low_Range" name="Low_Range"><br>
            <label for="High_Range">High_Range:</label>

```

```
        <input type="number" id="High_Range" name="High_Range"><br>
        <button type="submit">Add Packages</button>
    </form>
</div>
</div>
```

```
<div id="updatePackagesModal" class="modal">
    <div class="modal-content">
        <span class="close"
onclick="closeModal('updatePackagesModal')">&times;</span>
        <h2>Update Packages Information</h2>
        <form action="/update_Packages" method="POST">
            <label for="PackagesName">Code:</label>
            <input type="number" id="PackagesName" name="PackagesName"><br>
            <label for="Low_Range">Low_Range:</label>
            <input type="number" id="Low_Range" name="Low_Range"><br>
            <label for="High_Range">High_Range:</label>
            <input type="number" id="High_Range" name="High_Range"><br>
            <button type="submit">Update Packages</button>
        </form>
    </div>
</div>
```

```
<div id="deletePackagesModal" class="modal">
    <div class="modal-content">
        <span class="close"
onclick="closeModal('deletePackagesModal')">&times;</span>
        <h2>Delete Packages</h2>
        <form action="/delete_Packages" method="POST">
```

```

        <label for="PackagesName">Code:</label>
        <input type="number" id="PackagesName" name="PackagesName"><br>
        <button type="submit">Delete Packages</button>
    </form>
</div>
</div>

<div id="viewPackagesModal" class="modal">
    <div class="modal-content">
        <span class="close"
onclick="closeModal('viewPackagesModal')">&times;</span>
        <h2>View Packages Profile</h2>
        <form action="/view_Packages" method="POST">
            <label for="PackagesName">Code:</label>
            <input type="number" id="PackagesName" name="PackagesName"><br>
            <button type="submit">View Packages</button>
        </form>
    </div>
</div>

<!-- Modals for Client Management -->
<div id="addClientModal" class="modal">
    <div class="modal-content">
        <span class="close" onclick="closeModal('addClientModal')">&times;</span>
        <h2>Add Client</h2>
        <form action="/add_client" method="POST">
            <label for="clientId">Client ID:</label>
            <input type="text" id="clientId" name="clientId"><br>
            <label for="clientName">Name:</label>

```

```
<input type="text" id="clientName" name="clientName"><br>
<label for="clientEmail">Email:</label>
<input type="email" id="clientEmail" name="clientEmail"><br>
<label for="clientPhone">Phone:</label>
<input type="text" id="clientPhone" name="clientPhone"><br>
<button type="submit">Add Client</button>
</form>
</div>
</div>

<div id="updateClientModal" class="modal">
  <div class="modal-content">
    <span class="close" onclick="closeModal('updateClientModal')">&times;</span>
    <h2>Update Client Information</h2>
    <form action="/update_client" method="POST">
      <label for="clientId">Client ID:</label>
      <input type="text" id="clientId" name="clientId"><br>
      <label for="clientName">Name:</label>
      <input type="text" id="clientName" name="clientName"><br>
      <label for="clientEmail">Email:</label>
      <input type="email" id="clientEmail" name="clientEmail"><br>
      <label for="clientPhone">Phone:</label>
      <input type="text" id="clientPhone" name="clientPhone"><br>
      <button type="submit">Update Client</button>
    </form>
  </div>
</div>
</div>
```

```
<div id="deleteClientModal" class="modal">
  <div class="modal-content">
    <span class="close" onclick="closeModal('deleteClientModal')">&times;</span>
    <h2>Delete Client</h2>
    <form action="/delete_client" method="POST">
      <label for="clientId">Client ID:</label>
      <input type="text" id="clientId" name="clientId"><br>
      <button type="submit">Delete Client</button>
    </form>
  </div>
</div>
```

```
<div id="viewClientModal" class="modal">
  <div class="modal-content">
    <span class="close" onclick="closeModal('viewClientModal')">&times;</span>
    <h2>View Client Profile</h2>
    <form action="/view_client" method="POST">
      <label for="clientId">Client ID:</label>
      <input type="text" id="clientId" name="clientId"><br>
      <button type="submit">View Client</button>
    </form>
  </div>
</div>
```

<!-- Modals for Event Management -->

```
<div id="createEventModal" class="modal">
  <div class="modal-content">
    <span class="close" onclick="closeModal('createEventModal')">&times;</span>
```

<h2>Create Event</h2>

<form action="/create_event" method="POST">

<label for="eventId">Event Id:</label>

<input type="text" id="eventId" name="eventId">

<label for="eventName">Event Name:</label>

<input type="text" id="eventName" name="eventName">

<label for="eventDate">Event Date:</label>

<input type="date" id="eventDate" name="eventDate">

<label for="eventLocation">Event Desc:</label>

<input type="text" id="eventLocation" name="eventLocation">

<label for="assignedDj">Cost:</label>

<input type="text" id="assignedDj" name="assignedPackages">

<label for="eventVenue">Event Venue:</label>

<input type="date" id="eventVenue" name="eventVenue">

<label for="PACKAGE_CODE">PACKAGE CODE:</label>

<input type="text" id="PACKAGE_CODE" name="PACKAGE_CODE">

<label for="THEME_CODE">THEME CODE:</label>

<input type="text" id="THEME_CODE" name="THEME_CODE">

<label for="CLIENT_NUMBER">CLIENT NUMBER:</label>

<input type="text" id="CLIENT_NUMBER" name="CLIENT_NUMBER">

<button type="submit">Create Event</button>

</form>

</div>

</div>

<div id="updateEventModal" class="modal">

<div class="modal-content">

×

<h2>Update Event</h2>

<form action="/update_event" method="POST">

<label for="eventId">Event Id:</label>

<input type="text" id="eventId" name="eventId">

<label for="eventName">Event Name:</label>

<input type="text" id="eventName" name="eventName">

<label for="eventDate">Event Date:</label>

<input type="date" id="eventDate" name="eventDate">

<label for="eventLocation">Event Desc:</label>

<input type="text" id="eventLocation" name="eventLocation">

<label for="assignedDj">Cost:</label>

<input type="text" id="assignedDj" name="assignedPackages">

<label for="eventVenue">Event Venue:</label>

<input type="date" id="eventVenue" name="eventVenue">

<label for="PACKAGE_CODE">PACKAGE CODE:</label>

<input type="text" id="PACKAGE_CODE" name="PACKAGE_CODE">

<label for="THEME_CODE">THEME CODE:</label>

<input type="text" id="THEME_CODE" name="THEME_CODE">

<label for="CLIENT_NUMBER">CLIENT NUMBER:</label>

<input type="text" id="CLIENT_NUMBER" name="CLIENT_NUMBER">

<button type="submit">Update Event</button>

</form>

</div>

</div>

<div id="cancelEventModal" class="modal">

<div class="modal-content">

×

```

    <h2>Cancel Event</h2>
    <form action="/cancel_event" method="POST">
        <label for="eventId">Event ID:</label>
        <input type="text" id="eventId" name="eventId"><br>
        <button type="submit">Cancel Event</button>
    </form>
</div>
</div>

<div id="viewEventModal" class="modal">
    <div class="modal-content">
        <span class="close" onclick="closeModal('viewEventModal')">&times;</span>
        <h2>View Event Details</h2>
        <form action="/view_event" method="POST">
            <label for="eventId">Event ID:</label>
            <input type="text" id="eventId" name="eventId"><br>
            <button type="submit">View Event</button>
        </form>
    </div>
</div>

<!-- Modals for Booking Management -->
<div id="newBookingModal" class="modal">
    <div class="modal-content">
        <span class="close" onclick="closeModal('newBookingModal')">&times;</span>
        <h2>New Theme</h2>
        <form action="/new_booking" method="POST">
            <label for="bookingId">Theme ID:</label>

```

```
        <input type="text" id="bookingId" name="bookingId"><br>
        <label for="bookingClient">Theme Desc:</label>
        <input type="text" id="bookingClient" name="bookingClient"><br>
        <button type="submit">Create Theme</button>
    </form>
</div>
</div>
```

```
<div id="updateBookingModal" class="modal">
    <div class="modal-content">
        <span class="close"
onclick="closeModal('updateBookingModal')">&times;</span>
        <h2>Update Theme</h2>
        <form action="/update_booking" method="POST">
            <label for="bookingId">Theme ID:</label>
            <input type="text" id="bookingId" name="bookingId"><br>
            <label for="bookingClient">Theme Desc:</label>
            <input type="text" id="bookingClient" name="bookingClient"><br>
            <button type="submit">Update Theme</button>
        </form>
    </div>
</div>
```

```
<div id="cancelBookingModal" class="modal">
    <div class="modal-content">
        <span class="close"
onclick="closeModal('cancelBookingModal')">&times;</span>
        <h2>Delete Theme</h2>
        <form action="/cancel_booking" method="POST">
```

```

        <label for="bookingId">Theme ID:</label>
        <input type="text" id="bookingId" name="bookingId"><br>
        <button type="submit">Cancel Theme</button>
    </form>
</div>
</div>

<div id="viewBookingModal" class="modal">
    <div class="modal-content">
        <span class="close" onclick="closeModal('viewBookingModal')">&times;</span>
        <h2>View Theme Details</h2>
        <form action="/view_booking" method="POST">
            <label for="bookingId">Theme ID:</label>
            <input type="text" id="bookingId" name="bookingId"><br>
            <button type="submit">View Theme</button>
        </form>
    </div>
</div>

<!-- Modals for CD Management -->
<div id="addCDModal" class="modal">
    <div class="modal-content">
        <span class="close" onclick="closeModal('addCDModal')">&times;</span>
        <h2>Add CD</h2>
        <form action="/add_CD" method="POST">
            <label for="CDId">CD ID:</label>
            <input type="text" id="CDId" name="CDId"><br>
            <label for="CDClient">Title:</label>

```

```

        <input type="text" id="CDClient" name="CDClient"><br>
        <label for="CDAmount">Year:</label>
        <input type="number" id="CDAmount" name="CDAmount"><br>
        <button type="submit">Add CD</button>
    </form>
</div>
</div>
<div id="updateCDModal" class="modal">
    <div class="modal-content">
        <span class="close" onclick="closeModal('updateCDModal')">&times;</span>
        <h2>Update CD</h2>
        <form action="/update_CD" method="POST">
            <label for="CDId">CD ID:</label>
            <input type="text" id="CDId" name="CDId"><br>
            <label for="CDClient">Title:</label>
            <input type="text" id="CDClient" name="CDClient"><br>
            <label for="CDAmount">Year:</label>
            <input type="number" id="CDAmount" name="CDAmount"><br>
            <button type="submit">Update CD</button>
        </form>
    </div>
</div>
<div id="deleteCDModal" class="modal">
    <div class="modal-content">
        <span class="close" onclick="closeModal('deleteCDModal')">&times;</span>
        <h2>Delete CD</h2>
        <form action="/delete_CD" method="POST">

```

```

        <label for="CDId">CD ID:</label>
        <input type="text" id="CDId" name="CDId"><br>
        <button type="submit">Delete CD</button>
    </form>
</div>
</div>

<div id="viewCDModal" class="modal">
    <div class="modal-content">
        <span class="close" onclick="closeModal('viewCDModal')">&times;</span>
        <h2>View CD History</h2>
        <form action="/view_CD" method="POST">
            <label for="clientId">CD ID:</label>
            <input type="text" id="clientId" name="clientId"><br>
            <button type="submit">View CD History</button>
        </form>
    </div>
</div>

<script src="static/script.js"></script>
</body>
</html>

```

success.html

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>404 Not Found</title>
<link rel="stylesheet" href="static/sucess.css">
</head>
<body>
  <div class="success-animation">
    <svg class="checkmark" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 52
52"><circle class="checkmark__circle" cx="26" cy="26" r="25" fill="none" /><path
class="checkmark__check" fill="none" d="M14.1 27.2l7.1 7.2 16.7-16.8" /></svg>
    <div class="container">
      <div class="error">
        <p>Database Operation Completed Successfully.</p>
        <a href="/" class="home-link">Go Home</a>
      </div>
    </div>
  </div>
</body>
</div>
</html>
```

404.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>404 Not Found</title>
  <link rel="stylesheet" href="static/404.css">
</head>
```

```
<body>
  <div class="container">
    <div class="error">
      <h1>404</h1>
      <p>Entry Not Found.</p>
      <a href="/" class="home-link">Go Home</a>
    </div>
  </div>
</body>
</html>
```

view.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>User Info</title>
  <link rel="stylesheet" href="global.css">
</head>
<body>
  <div class="container">
    <h1>User Information</h1>
    <p><strong>Name:</strong> {{ data.name }}</p>
    <p><strong>Phone:</strong> {{ data.phone }}</p>
    <p><strong>ID:</strong> {{ data.id }}</p>
    <p><strong>Email:</strong> {{ data.email }}</p>
  </div>
```

```
</body>
```

```
</html>
```

viewCd.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>CD Information</title>
```

```
  <link rel="stylesheet" href="global.css">
```

```
</head>
```

```
<body>
```

```
  <div class="container">
```

```
    <h1>CD Information</h1>
```

```
    <p><strong>ID:</strong> {{ data.id }}</p>
```

```
    <p><strong>Year:</strong> {{ data.year }}</p>
```

```
    <p><strong>title:</strong> {{ data.title }}</p>
```

```
  </div>
```

```
</body>
```

```
</html>
```

viewEvent.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Event Info</title>
<link rel="stylesheet" href="global.css">
</head>
<body>
  <div class="container">
    <h1>Event Information</h1>
    <p><strong>Event Id:</strong> {{ data.id }}</p>
    <p><strong>Name:</strong> {{ data.name }}</p>
    <p><strong>Date:</strong> {{ data.date }}</p>
    <p><strong>Desc:</strong> {{ data.desc }}</p>
    <p><strong>Cost:</strong> {{ data.cost }}</p>
    <p><strong>Venue:</strong> {{ data.venue }}</p>
    <p><strong>Package code:</strong> {{ data.pack }}</p>
    <p><strong>Theme Coded:</strong> {{ data.theme }}</p>
    <p><strong>Client Number:</strong> {{ data.client }}</p>
  </div>
</body>
</html>
```

viewPack.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>User Info</title>
  <link rel="stylesheet" href="global.css">
</head>
```

```
<body>
  <div class="container">
    <h1>User Information</h1>
    <p><strong>Code:</strong> {{ data.code }}</p>
    <p><strong>Low_Range:</strong> {{ data.low }}</p>
    <p><strong>High_Range:</strong> {{ data.high }}</p>

  </div>
</body>
</html>
```

viewTheme.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Theme Info</title>
  <link rel="stylesheet" href="global.css">
</head>
<body>
  <div class="container">
    <h1>Theme Information</h1>
    <p><strong>ID:</strong> {{ data.id }}</p>
    <p><strong>Desc:</strong> {{ data.desc }}</p>
  </div>
</body>
</html>
```

style.css

```
/* static/style.css */
```

```
body {  
    font-family: Arial, sans-serif;  
    background-color: #e0f7fa;  
    margin: 0;  
    padding: 0;  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    flex-direction: column;  
    height: 100vh;  
}  
  
h1{  
    font-size: 55px;  
    color: #45a049;  
    filter: drop-shadow(3px 2px 2px #000000);  
}  
  
.container {  
    width: 80%;  
    background: #ffffff;  
    border-radius: 8px;  
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);  
    padding: 20px;  
}  
  
.tabs {  
    display: flex;
```

```
    justify-content: space-around;
    border-bottom: 1px solid #ccc;
}

.tablinks {
    background-color: inherit;
    border: none;
    outline: none;
    cursor: pointer;
    padding: 14px 16px;
    transition: 0.3s;
    font-size: 17px;
}

.tablinks:hover {
    background-color: #ddd;
}

.tablinks.active {
    background-color: #ccc;
}

.tabcontent {
    display: none;
    padding: 20px;
    border-top: none;
}

.tabcontent button {
    background-color: #4CAF50;
    border: none;
    color: white;
```

```
padding: 10px 24px;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
margin: 4px 2px;
cursor: pointer;
border-radius: 4px;
}
.tabcontent button:hover {
    background-color: #45a049;
}
/* Modal styles */
.modal {
    display: none;
    position: fixed;
    z-index: 1;
    left: 0;
    top: 0;
    width: 100%;
    height: 100%;
    overflow: auto;
    background-color: rgb(0, 0, 0);
    background-color: rgba(0, 0, 0, 0.4);
    padding-top: 60px;
}
.modal-content {
    background-color: #fefefe;
```

```
margin: 5% auto;
padding: 20px;
border: 1px solid #888;
width: 40%;
border-radius: 8px;
}
.close {
  color: #aaa;
  float: right;
  font-size: 28px;
  font-weight: bold;
}
.close:hover,
.close:focus {
  color: black;
  text-decoration: none;
  cursor: pointer;
}
form {
  display: flex;
  flex-direction: column;
}
form label, form input, form button {
  margin: 5px 0;
}
/* Add these styles for dialog boxes */
.dialog {
  display: none;
```

```
position: fixed;
z-index: 1;
left: 0;
top: 0;
width: 100%;
height: 100%;
overflow: auto;
background-color: rgba(0, 0, 0, 0.4);
padding-top: 60px;
}
.dialog-content {
background-color: #fefefe;
margin: 15% auto;
padding: 20px;
border: 1px solid #888;
width: 80%;
text-align: center;
}
#viewPage {
display: none;
padding: 20px;
}
#viewContent {
margin-top: 20px;
}
```

404.css

```
body, html {
```

```
margin: 0;
padding: 0;
width: 100%;
height: 100%;
display: flex;
justify-content: center;
align-items: center;
background: #000;
font-family: 'Arial', sans-serif;
color: #fff;
}

.container {
  position: relative;
  text-align: center;
}

.error h1 {
  font-size: 10rem;
  margin: 0;
}

.error p {
  font-size: 1.5rem;
}

.home-link {
  display: inline-block;
  margin-top: 20px;
  padding: 10px 20px;
  color: #000;
  background: #fff;
```

```
text-decoration: none;
border-radius: 5px;
transition: background 0.3s;
}
.home-link:hover {
  background: #ddd;
}
.animated-background {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  overflow: hidden;
  z-index: -1;
}
```

success.css

```
.success-animation { margin:150px auto;}
.checkmark {
  width: 100px;
  height: 100px;
  border-radius: 50%;
  display: block;
  stroke-width: 2;
  stroke: #4bb71b;
  stroke-miterlimit: 10;
  box-shadow: inset 0px 0px 0px #4bb71b;
```

```
    animation: fill .4s ease-in-out .4s forwards, scale .3s ease-in-out .9s both;
    position: relative;
    top: 5px;
    right: 5px;
    margin: 0 auto;
}

.checkmark__circle {
    stroke-dasharray: 166;
    stroke-dashoffset: 166;
    stroke-width: 2;
    stroke-miterlimit: 10;
    stroke: #4bb71b;
    fill: #fff;
    animation: stroke 0.6s cubic-bezier(0.65, 0, 0.45, 1) forwards;
}

.checkmark__check {
    transform-origin: 50% 50%;
    stroke-dasharray: 48;
    stroke-dashoffset: 48;
    animation: stroke 0.3s cubic-bezier(0.65, 0, 0.45, 1) 0.8s forwards;
}

@keyframes stroke {
    100% {
        stroke-dashoffset: 0;
    }
}

@keyframes scale {
    0%, 100% {
```

```
        transform: none;
    }
    50% {
        transform: scale3d(1.1, 1.1, 1);
    }
}
@keyframes fill {
    100% {
        box-shadow: inset 0px 0px 0px 30px #4bb71b;
    }
}
body, html {
    margin: 0;
    padding: 0;
    width: 100%;
    height: 100%;
    display: flex;
    justify-content: center;
    align-items: center;
    font-family: 'Arial', sans-serif;
    color: #fff;
}
.container {
    margin-top: 5vh;
    position: relative;
    text-align: center;
}
.error h1 {
```

```
    font-size: 10rem;
    margin: 0;
}
.error p {
    color: black;
    font-size: 2rem;
}
.home-link {
    display: inline-block;
    margin-top: 20px;
    padding: 10px 20px;
    color: #000;
    background: #4bb71b;
    text-decoration: none;
    border-radius: 5px;
    transition: background 0.3s;
}
.home-link:hover {
    background: #317015;
}
.animated-background {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    overflow: hidden;
    z-index: -1;
```

```
}
```

global.css

```
body {
```

```
    display: flex;
```

```
    justify-content: center;
```

```
    align-items: center;
```

```
    height: 100vh;
```

```
    margin: 0;
```

```
    font-family: Arial, sans-serif;
```

```
}
```

```
.container {
```

```
    text-align: center;
```

```
    border: 1px solid #ccc;
```

```
    padding: 20px;
```

```
    border-radius: 10px;
```

```
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
```

```
}
```

```
.container h1 {
```

```
    margin-bottom: 20px;
```

```
}
```

```
.container p {
```

```
    margin: 5px 0;
```

```
}
```

script.js

```
document.addEventListener('DOMContentLoaded', (event) => {
```

```
    document.getElementById("defaultOpen").click();
```

```
});
```

```
function openTab(evt, tabName) {
```

```
var i, tabcontent, tablinks;

tabcontent = document.getElementsByClassName("tabcontent");
for (i = 0; i < tabcontent.length; i++) {
    tabcontent[i].style.display = "none";
}

tablinks = document.getElementsByClassName("tablinks");
for (i = 0; i < tablinks.length; i++) {
    tablinks[i].className = tablinks[i].className.replace(" active", "");
}

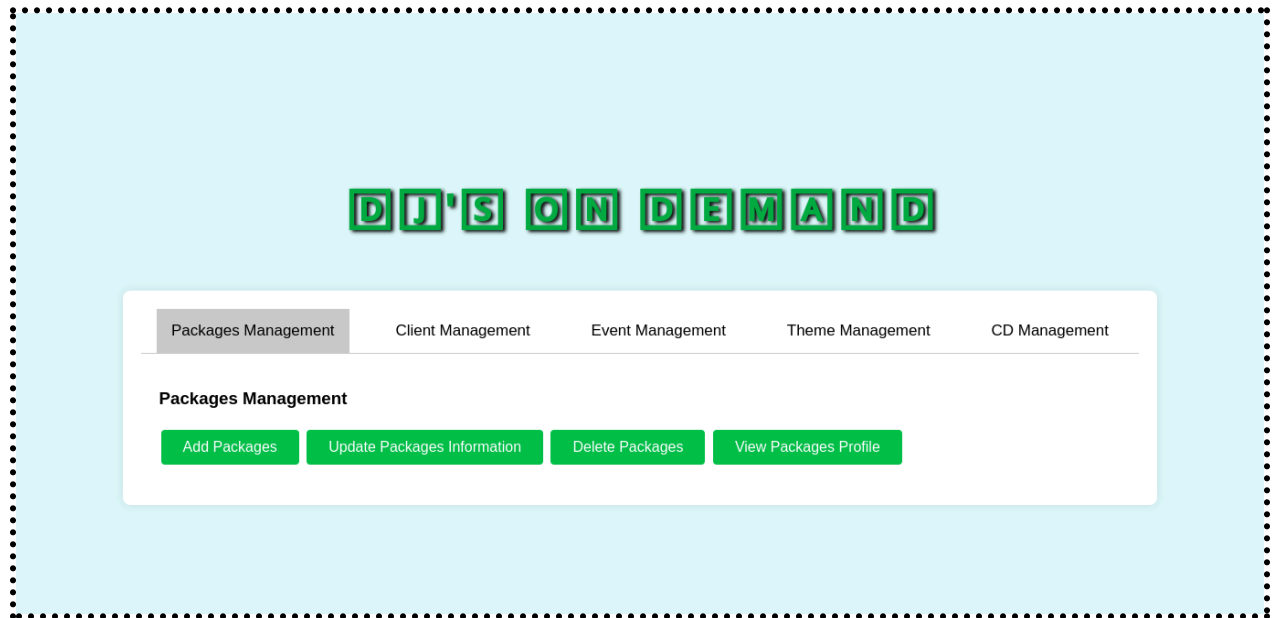
document.getElementById(tabName).style.display = "block";
evt.currentTarget.className += " active";
}

function openModal(modalId) {
    document.getElementById(modalId).style.display = "block";
}

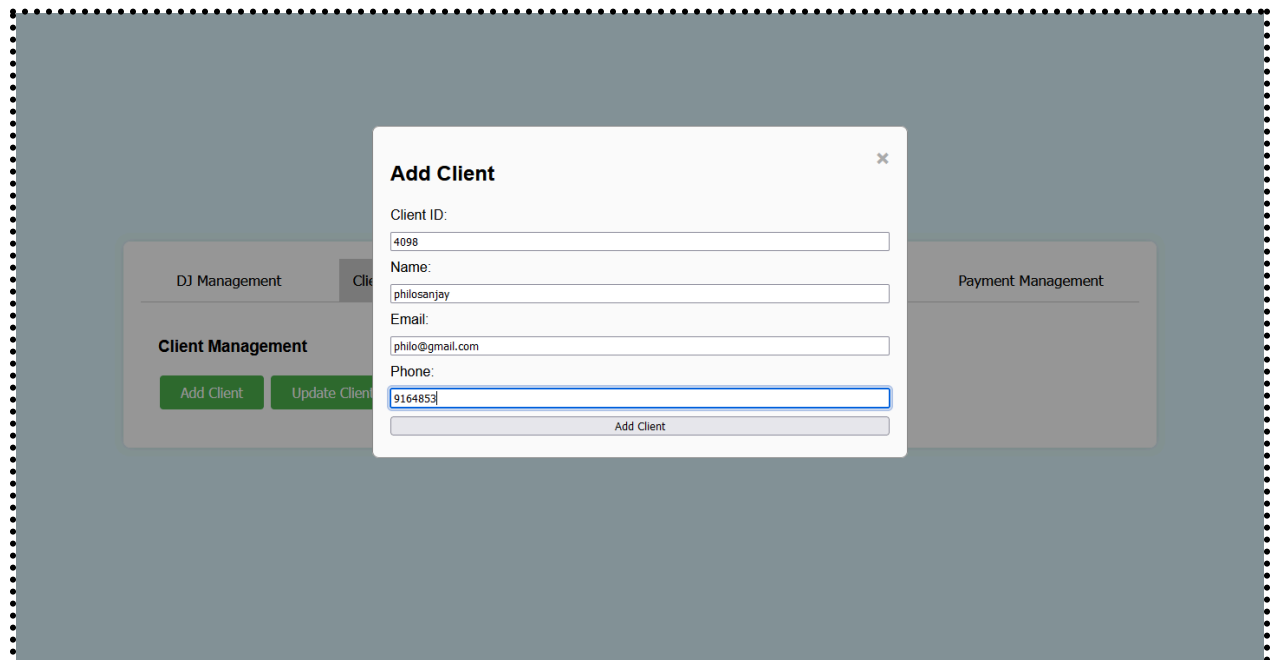
function closeModal(modalId) {
    document.getElementById(modalId).style.display = "none";
}
```

OUTPUT:

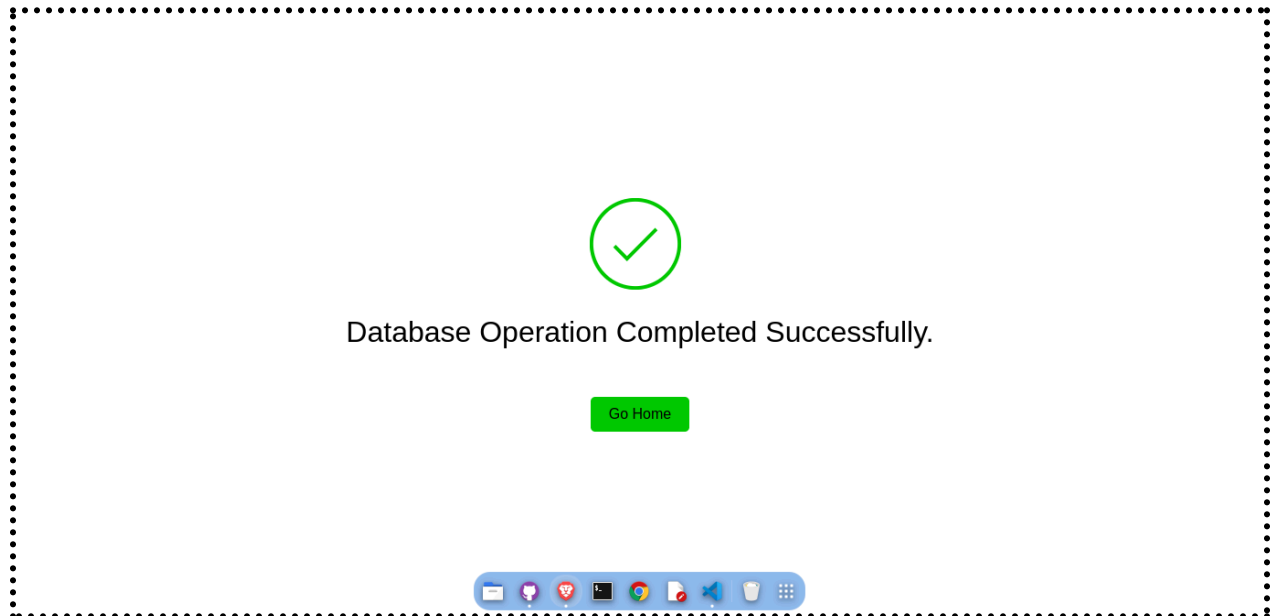
-- Homepage



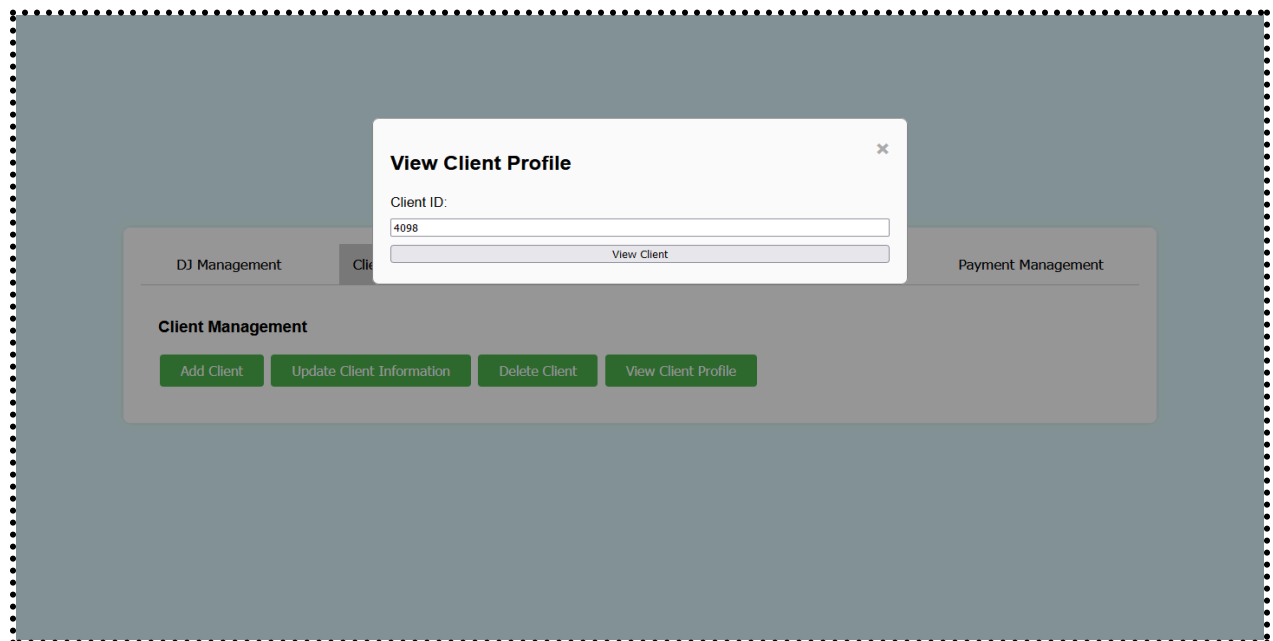
-- Add Client



-- Data Added Successfully



-- View added client



-- View result

User Information

Name: philosanjay philosanjay

Phone: 9164853

ID: 4098

Email: philo@gmail.com

-- Update the value of the entry

DJ Management

Client Management

Add Client

Update Client

Update Client Information

Client ID:

4098

Name:

philo change

Email:

philo@hotmail

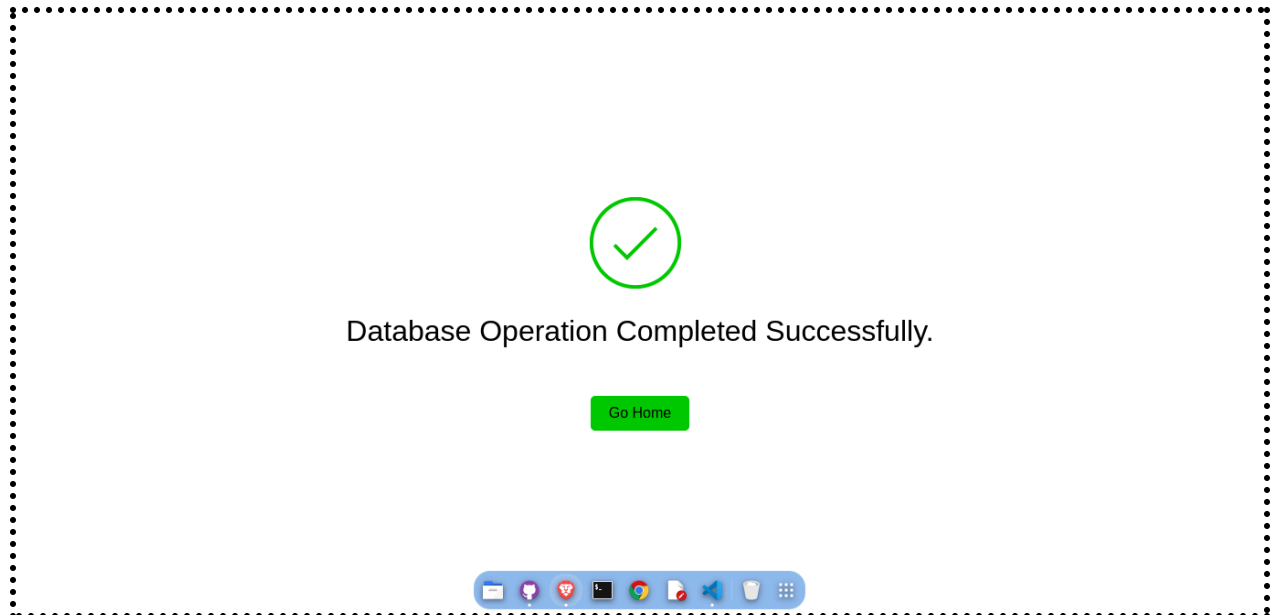
Phone:

123456789

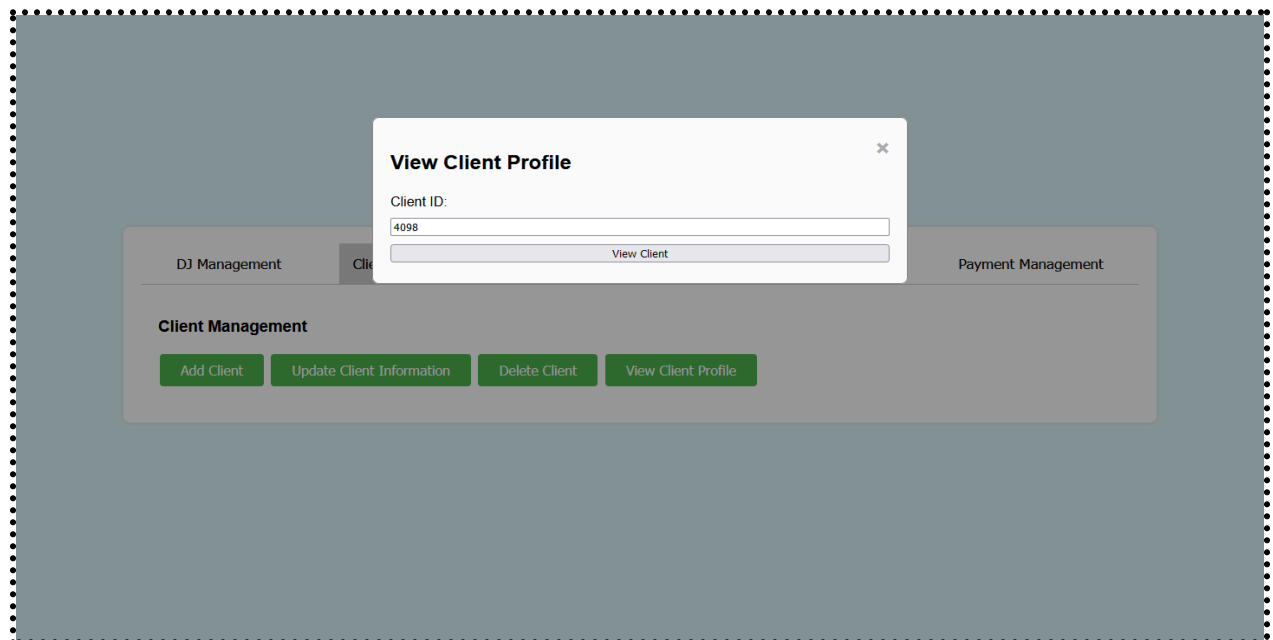
Update Client

Payment Management

-- Data Updated Successfully



-- Access Updated Values



-- Updated Values

User Information

Name: philo change philosanjay

Phone: 123456789

ID: 4098

Email: philo@hotmail

-- Delete Entry Client

DJ Management

Client Management

Payment Management

Add Client

Update Client Information

Delete Client

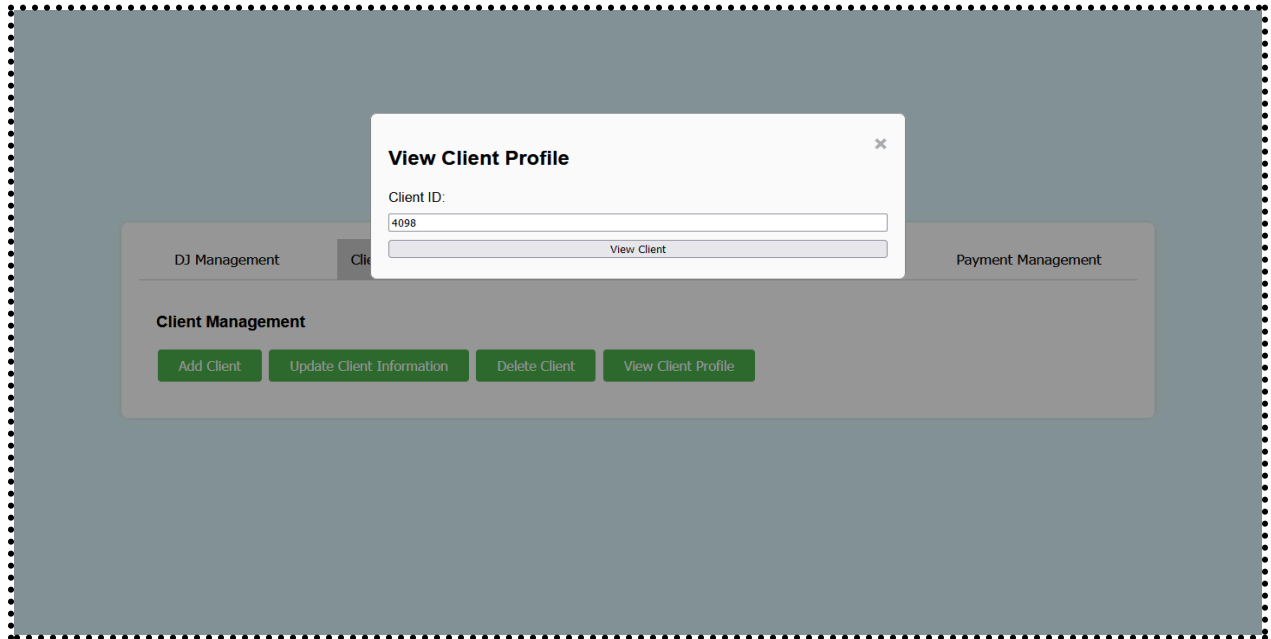
View Client Profile

Delete Client

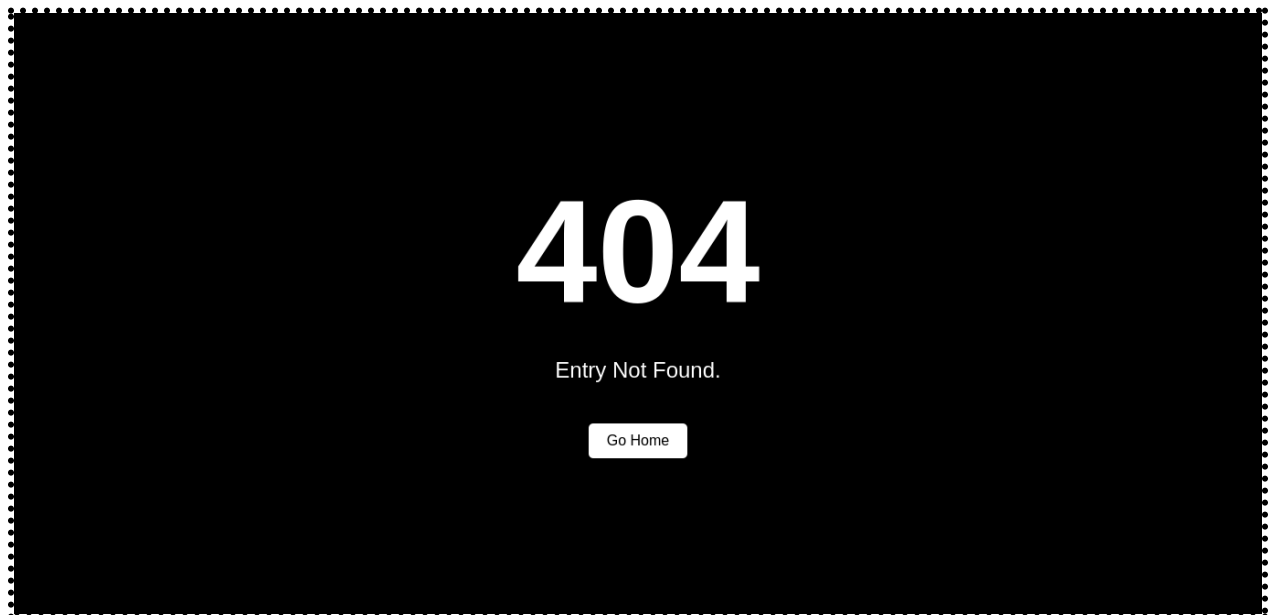
Client ID:
4098

Delete Client

-- Access Deleted Entry



-- Missing value page



Result:

Thus the creation of a real time database application by designing an ER diagram, applying normalization principles on the entities and incorporating the necessary SQL and PL/SQL features were successfully built, executed and verified.